

Dossier de projet
Boutique en ligne
Sac'Azura

Développé par Barbey
Mathieu
Pour le compte de la micro-entreprise
Sac'Azura

Centre Européen de formation
Années 2023-2026

Remerciements.....	2
I/ Présentation du projet.....	3
II/ Analyse et conception.....	4
II.a/ Conception et technologies.....	4
II.a.1/ Le backend.....	4
II.a.2/ Le frontend.....	4
II.a.3/ La sécurité.....	4
II.a.4/ Le SEO.....	4
II.a.5/ Versionning et déploiement.....	5
II.a.6/ Le RGPD.....	5
II.b/ Besoins et solutions.....	5
III/ Développement.....	10
3.1/ Les étapes du développement.....	10
3.1.a/ Le backend.....	10
3.1.b/ Le frontend.....	11
3.2/ Le code source.....	11
3.2.a/ backend.....	11
3.2.a.1/ La base de données.....	12
3.2.a.2/ L'authentification.....	13
3.2.a.3/ La gestion des produits et des stocks.....	17
3.2.a.4/ La gestion des images.....	21
3.2.a.5/ La gestion des stocks.....	22
3.2.a.6/ Le paiement.....	24
3.2.a.7/ Gestion des commandes.....	28
3.2.a.8/ Gestion des expéditions.....	29
3.2.a.9/ Gestion du CSRF.....	32
3.2.b/ Le frontend.....	33
IV/ Test et validation.....	36
4.1/ Test unitaire.....	36
4.2/ Test fonctionnel.....	38
4.3/ Test d'intégration.....	39
V/ Documentation.....	39
VI/ Bilan personnel.....	39
Annexe.....	40

Remerciements

Je tiens tout d'abord à exprimer ma gratitude envers mon mentor Fahed Mahidi qui m'a guidé tout au long de ma formation, mais qui m'aura aussi permis, au fil de de session, d'aller plus loin que ce que proposait juste la formation. Je n'aurais pas pu aller aussi loin dans mes projets sans ses conseils.

Mes remerciements vont aussi à mes amis eux-même developpeurs web et qui m'auront apporté de l'aide dans les moments ou je ne trouvais de solution a un problème, ou qui m'auront apporté une expertise sur des sujets que je ne connaissais pas. C'est aussi grâce à eux que je me suis dirigé vers le développement web.

Je remercie le jury qui prendra le temps de lire ce dossier présentant ce projet qui m'a demandé énormément de temps mais qui m'aura aussi énormément appris.

Je remercie aussi mes amis qui m'ont apporté leur soutien, mais aussi de m'avoir accompagné durant mes après-midi de codage durant lesquels nous travaillions sur nos propres projets tout en échangeant des idées autours de ces mêmes projets.

Enfin, je tiens à remercier Nadège Alain, auto-entrepreneuse, qui m'a donné la possibilité de réaliser ce projet pour terminer ma formation.

Ce projet est le fruit de mon propre travail, mais aussi le fruit des échanges et de l'apport de beaucoup de personnes, et je suis reconnaissant envers toutes les personnes qui m'ont accompagné.

I/ Présentation du projet

Ce projet de boutique en ligne est né de deux besoins. Le premier besoin est une boutique en ligne pour une personne en auto-entreprise dont le nom est Sac'Azura. Le second est mon besoin d'un projet concret pour valider ma formation. Au cours de celui-ci, une troisième personne s'est jointe à nous. Cette personne, qui est la fille de la propriétaire de Sac'Azura, est actuellement en formation en marketing digital et avait aussi besoin d'un projet pour valider celle-ci. Information importante à propos de ce projet, celui-ci n'est pas terminé, mais le terminer ne serait pas possible avant la fin de la formation. Je vais donc présenter un démonstrateur de ce que le site proposera. Certaines pages retourneront des pages 404 not found dû à cette raison. Aussi, je vais continuer d'avancer dans le projet après le rendu du dossier projet. Il y aura donc certainement des éléments supplémentaires par rapport à ce que le dossier proposé, et ce qui sera observable sur le github.

Sac'Azura est une entreprise d'upcycling faisant principalement des accessoires tels que des boîtiers à lunette ou poche à mouchoir, des sacs tels que des sacs à dos ou des sac a main, ou encore des gants de tout genre. Le upcycling est un concept relativement récent. Celui-ci s'appuie sur la réutilisation de matériaux pour créer de nouvelles choses. Ce concept est de plus en plus présent dans le domaine du textile en particulier. En plus de cet aspect écologique, Sac'Azura s'appuie sur un aspect local très fort avec l'utilisation de matériaux récupérés uniquement dans le secteur de l'entreprise, ainsi que le travail avec des artisans locaux pour les éléments que la personne ne peut faire elle-même.

Pour ce projet, il a été décidé d'utiliser une base de données relationnelle MySQL. Celle-ci va être accessible à l'aide d'une API créée avec NestJs, un framework sous NodeJs. La partie frontend est faite avec le framework NextJs permettant d'exploiter les avantages de React pour produire une SPA (Single Page Application) en combinant avec le server rendering qui est un des principaux avantages de NextJs. En termes de spécificité technique, je vais lister l'ensemble de celle-ci sachant que plusieurs ne seront pas encore disponibles. La première spécificité est la création d'un compte et la connexion à celui-ci pour un utilisateur. Celui-ci aura accès à une page profil lui permettant de changer ses informations, voir ses adresses, ainsi que voir les commandes et leur statut. Les clients pourront suivre un processus classique d'achat. C'est-à-dire pouvoir chercher et/ou filtrer parmi une liste de produits. Ils pourront ensuite sélectionner la variante du produit en terme de couleur et/ou matériau et l'ajouter au panier en fonction de si le stock le permet. Ils pourront ensuite valider le panier redirigeant vers la page pour sélectionner ou ajouter une adresse et enfin, valider le paiement via l'API Stripe. C'est alors qu'ils pourraient avoir le suivi de la commande dans leur espace. Il leur sera aussi possible de demander pour une commande personnalisée. Pour gérer l'ensemble du site, une page administration accessible uniquement avec le statut administrateur permettra d'effectuer toutes les actions du CRUD (Create, Retrieve, Update, Delete) pour les catégories, sous-catégories, couleur, matériau, produit et utilisateur.

II/ Analyse et conception

Cette partie va décrire les besoins de ce projet, ainsi que donner les solutions utilisées pour y répondre. Je vais commencer par les besoins regroupant le projet dans sa globalité, puis je décrirai les besoins et solutions liées aux utilisateurs. Je terminerai avec ceux autour des produits et du processus d'achat.

II.a/ Conception et technologies

II.a.1/ Le backend

Le backend repose sur une API REST code à l'aide du framework NestJs. Ce framework se basant sous NodeJs. Ce framework apporte tous les avantages des frameworks de ce type. On retrouve, par exemple, l'utilisation de Typescript, ou encore une architecture inspirée par Angular qui impose donc une architecture stricte et plus facilement scalable dans le futur. Cet API permet de faire le lien entre le frontend et la base de données. J'ai décidé d'utiliser une base de données relationnelle donc en SQL avec comme raison principale la possibilité de créer des relations entre les tables.

II.a.2/ Le frontend

Le frontend repose sur le framework NextJs. Ce framework se basant lui-même sur React apporte certains avantages en terme de SEO tout en gardant les avantages de React comme le Single Page Application (SPA).

II.a.3/ La sécurité

Une grosse partie de la sécurité va reposer sur l'utilisation des DTO ou Data Transfer Object. Ces DTO permettent de vérifier et formater les données à l'aide des bibliothèques class-validator et class-transformer. C'est grâce à cela que l'application est protégée des injections SQL ou encore des attaques XSS car le DTO va bloquer les entrées en formulaire qui ne correspondent pas à ce qui est attendu. Ensuite, il faut sécuriser le chemin entre le frontend et le backend. Pour cela, CORS est configuré pour n'écouter que les appels depuis le frontend. En plus de cela, certaines routes, principalement les routes qui font appel à la base de données, sont protégées contre les attaques CSRF grâce à la mise en place d'un token CSRF.

II.a.4/ Le SEO

Le SEO du site va être un point important dans son positionnement. Pour que celui-ci soit au mieux, NextJs nous donne accès au Server Side Rendering (SSR) qui permet un pré-rendu des pages ce qui facilite le travail des robots d'indexation. En plus de cela, NestJs nous permet de définir les metadata pour chaque page grâce à la définition dans chaque

page d'une constante metadata. Cette constante peut contenir un titre, une description ou encore des mots clés.

II.a.5/ Versionning et déploiement

Le versionning s'est fait à l'aide de git et le site Github. Le projet n'a pas été déployé pour le moment, celui-ci étant toujours en développement, mais des solutions d'hébergement sont à l'étude avec la propriétaire de Sac'Azura. Pour le moment, celui-ci est déployé localement lors du développement.

II.a.6/ Le RGPD

Le site ne collecte aucune donnée de l'utilisateur sans que celui-ci ne renseigne d'information dans les formulaires tel que le formulaire d'inscription. Ces données sont ensuite gardées en base de données et uniquement accessibles par l'utilisateur ou un administrateur. La suppression du compte entraîne la suppression de toutes les informations de l'utilisateur hormis ses commandes pour garder une trace pour la gestion financière, mais celles-ci ne sont plus associées à un compte.

II.b/ Besoins et solutions

Pour peupler cette base de données, la propriétaire du site va avoir besoin de pouvoir voir ce que contient la base de données, mais elle va aussi avoir besoin de pouvoir créer, modifier et supprimer les catégories, les sous-catégories, les produits avec leur stock correspondant à chaque variante, mais aussi les différentes couleurs et matériaux qui déterminent les variantes d'un produit. Elle doit aussi pouvoir retrouver la commande d'un client pour intervenir dessus au besoin. Il faut donc créer un dashboard qui sera bloqué par un guard NestJs qui vérifiera si la personne est bien un administrateur ou non, et la laisser passer si c'est bien le cas.

Un élément de la base de données va demander une gestion plus poussée. Il s'agit des images. Chaque produit peut avoir une ou plusieurs images associées. Or, une image ne peut pas s'enregistrer dans une base de données SQL, du moins, cela est possible mais très peu recommandée face à des services dédiés. C'est pour cela que j'ai décidé d'utiliser un stockage en cloud, et plus précisément les buckets S3 proposés par les services AWS. Il suffit ensuite d'enregistrer le key de l'image en base de données. AWS permet aussi de sécuriser les images en générant une url temporaire pour accéder à chaque image. Le site doit être facilement utilisable depuis un smartphone. Celui-ci a donc été fait de manière à être responsive à l'aide de Tailwind.

Maintenant que le site est accessible, l'utilisateur doit pouvoir créer un compte et se connecter pour pouvoir acheter des produits. Deux formulaires permettent de répondre à ces besoins. Le premier est un formulaire d'inscription. Celui-ci va comporter les champs pour les nom, prénom, email, téléphone, mot de passe ainsi que la confirmation du mot de passe. Un DTO dans le backend va vérifier le format des données du formulaire avant que ces données soient enregistrées en base de données après un cryptage du mot de passe.

Suite à l'inscription, un mail avec un lien généré avec un token va demander à l'utilisateur de valider le compte ce qui lui donnera accès au paiement sur le site. Le second formulaire est le formulaire de connexion. Comme pour le premier formulaire, les données sont vérifiées par un DTO. Après que la connexion ait été validée, un token JWT est généré et enregistré dans les cookies. Ce token sera vérifié à chaque fois à l'aide de la bibliothèque Passport ainsi que d'un guard NestJs JwtAuthGuard. Pour centraliser sur toute la partie front de l'application l'authentification, un contexte React va regrouper les méthodes pour se connecter et se déconnecter.

L'utilisateur peut maintenant utiliser le site en étant connecté. Mais il faut éviter que celui-ci ait besoin de se reconnecter après chaque action, ou que la connexion ne dure indéfiniment entraînant un risque de sécurité. Pour répondre à cela, le token est stocké dans les cookies et envoyé à chaque call requérant que l'utilisateur soit connecté. Le guard NestJs va alors vérifier si l'utilisateur est bien connecté.

Dans le cas où l'utilisateur a oublié son mot de passe, il sera possible de demander un lien pour changer de mot de passe. Un email contenant un token sera alors envoyé. Ce lien mènera vers un nouveau formulaire qui permettra de changer le mot de passe. Un DTO vérifiera les informations et le mot de passe sera crypté avant d'être enregistré en base de données.

Pour pouvoir modifier ses informations ou encore suivre ses commandes, l'utilisateur aura besoin d'une page profil. Il pourra modifier les informations de son compte à l'aide d'un formulaire dont les données seront vérifiées par un DTO avant de modifier la base de données. Il pourra aussi faire de même pour ses adresses. Il lui sera aussi possible de voir ses commandes. L'utilisateur pourra aussi, depuis cette page, supprimer son compte à l'aide d'un bouton ce qui résulte sur la suppression de ses informations de la base de données. Maintenant que l'utilisateur peut créer son compte, se connecter et faire toutes les actions vues précédemment, celui-ci va vouloir acheter un produit. Il pourra alors se rendre sur la page boutique. La boutique propose différents types de gants et de sacs. L'utilisateur aura donc besoin de filtrer pour trouver ce qu'il cherche. Un menu déroulant lui permettra de choisir la sous-catégorie. L'application n'ayant pas une grande quantité de produit en même temps en base de données, le filtrage se fera directement sur le tableau de produit retourné depuis la base de données.

Après avoir repéré un produit, le client aura la volonté d'obtenir plus d'informations à propos de celui-ci. Un bouton lui permettra de se diriger vers la page du produit en utilisant une url avec comme paramètre l'identifiant du produit.

Chaque produit peut avoir des variantes en fonction de sa couleur et/ou de son matériau de fabrication. Chaque variante va avoir un stock attribué. Un utilisateur devra donc pouvoir choisir une variante, et avoir la possibilité ou non de l'ajouter au panier. Cela se traduit par un formulaire à choix permettant de choisir la couleur et/ou le matériau s'il y a du stock ou non.

Après avoir sélectionné une variante, l'utilisateur va alors l'ajouter au panier. Un bouton "Ajouter au panier" est présent, et va permettre d'utiliser un contexte React qui gère le panier, pour ajouter un produit dans celui-ci.

Dans le cas où l'utilisateur n'est pas connecté au moment d'ajouter le produit dans le panier, il pourra conserver son panier en se connectant. Le contexte sauvegardé dans le LocalStorage le contenu du panier. Celui-ci sera alors relié à l'utilisateur connecté.

L'utilisateur pourra alors accéder à son panier via un bouton dans le header pour en voir le contenu.

Si un code promotionnel est disponible, un formulaire est présent dans le panier pour tester si le code existe bien, est toujours valide et le prix après application du code..

Après avoir validé le panier, l'utilisateur devra ajouter une adresse de livraison. Un formulaire sera alors disponible avant le paiement lui permettant d'ajouter une adresse.

Dans le cas où une adresse est déjà disponible, celle-ci sera sélectionnable.

Suite à l'ajout de l'adresse, l'utilisateur voudra passer au paiement. Un bouton permet la redirection vers l'API Stripe pour faire l'achat. Si le compte n'est pas vérifié, une alerte apparaîtra proposant de renvoyer un mail de validation de compte.

Suite au paiement, les stocks doivent être mis à jour. Un webhook surveillant si l'API Stripe retourne une complétion de la session de paiement, va alors s'occuper de la mise à jour des stocks en récupérant les produits et les quantités achetées via les metadatas.

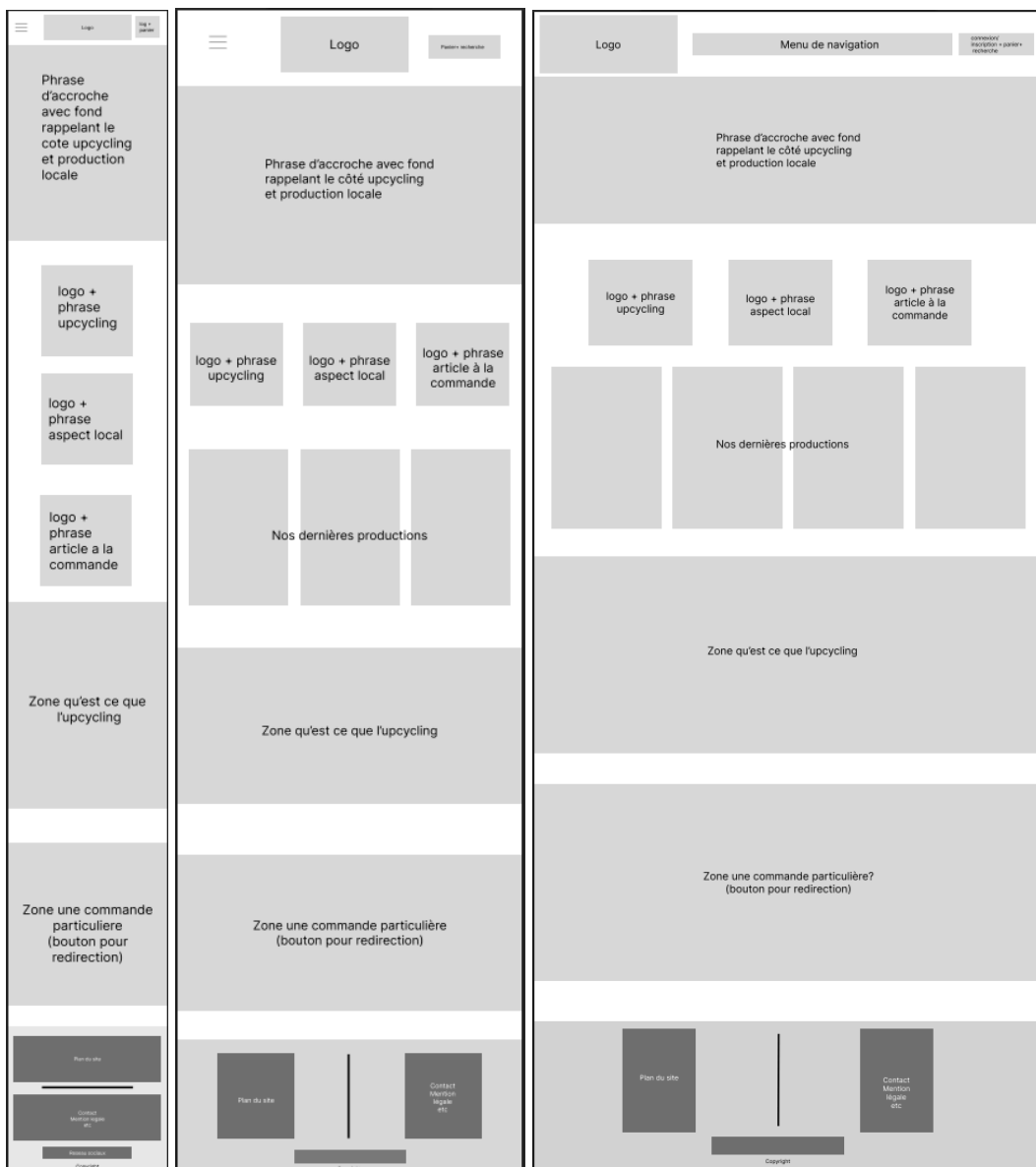
Après son achat, l'utilisateur doit savoir si l'achat a été un succès ou non. Pour le cas d'un succès, une redirection vers une page indiquant que le paiement est un succès, et un mail est automatiquement envoyé. Il en est de même pour le cas d'un échec de paiement. Une redirection vers une page d'échec est effectuée, et un mail indiquant l'échec du paiement est envoyé. Le client effectuant un paiement pour une commande, il est important de lui indiquer les frais de port ainsi que le statut de la commande. Pour gérer cet aspect, les services web de Mondial Relay et Colissimo sont implémentés pour permettre à l'utilisateur de choisir un point relais (dans le cas de Mondial Relay), de connaître les frais de port du colis, ainsi que d'avoir le suivi de la commande.

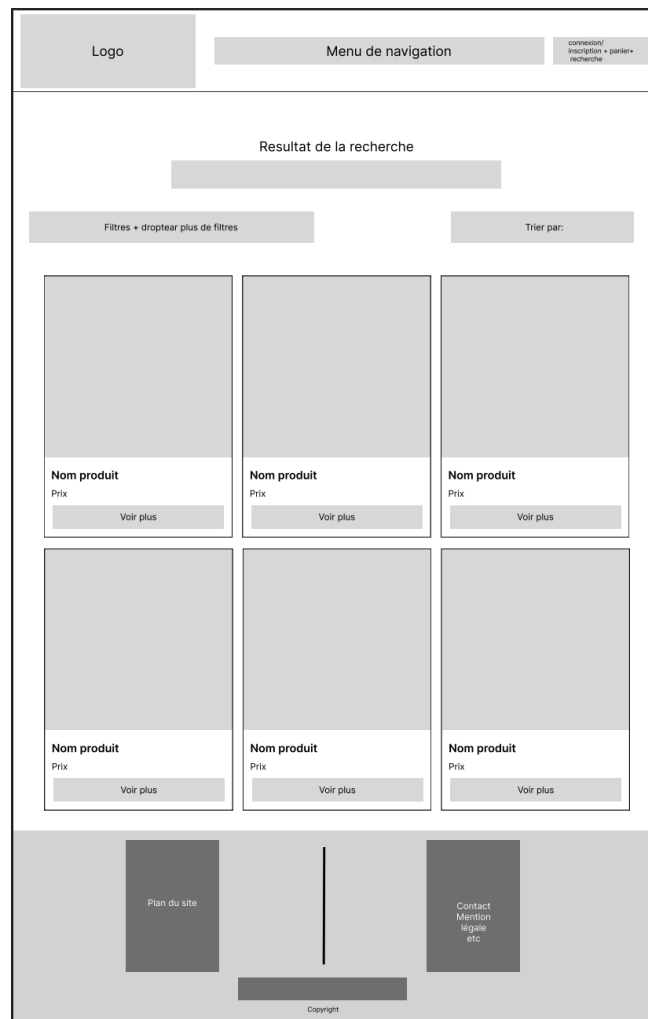
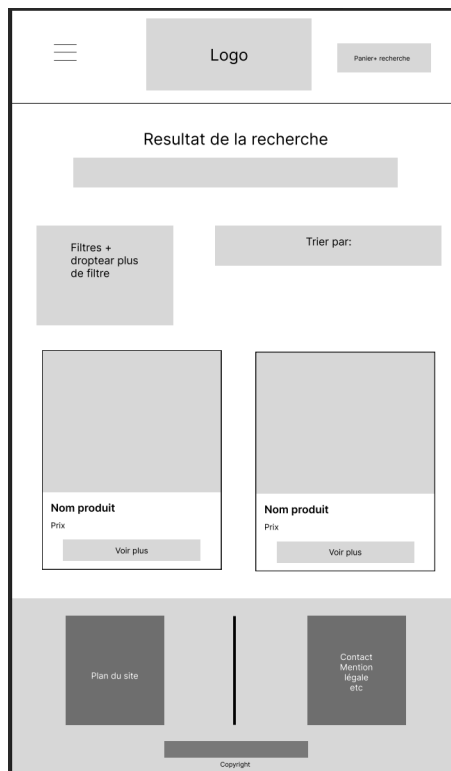
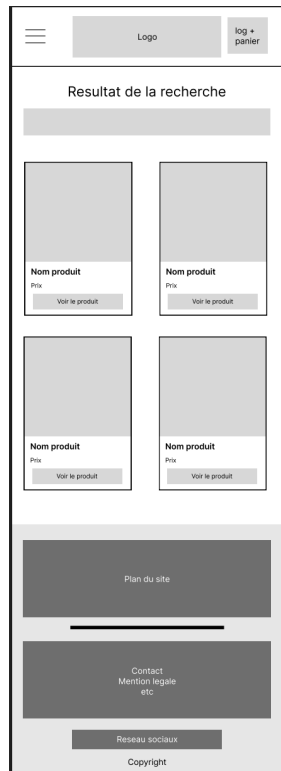
Il est aussi possible pour le client de faire une commande personnalisée. Pour se faire, une page dédiée avec un formulaire de contact spécifique pour ce besoin sera créée pour permettre de décrire le besoin.

Une newsletter sera disponible et il sera possible de s'y inscrire. Un formulaire sera disponible pour renseigner son mail. Après cela, un mail de confirmation sera envoyé pour valider l'ajout du mail sur la liste de la newsletter.

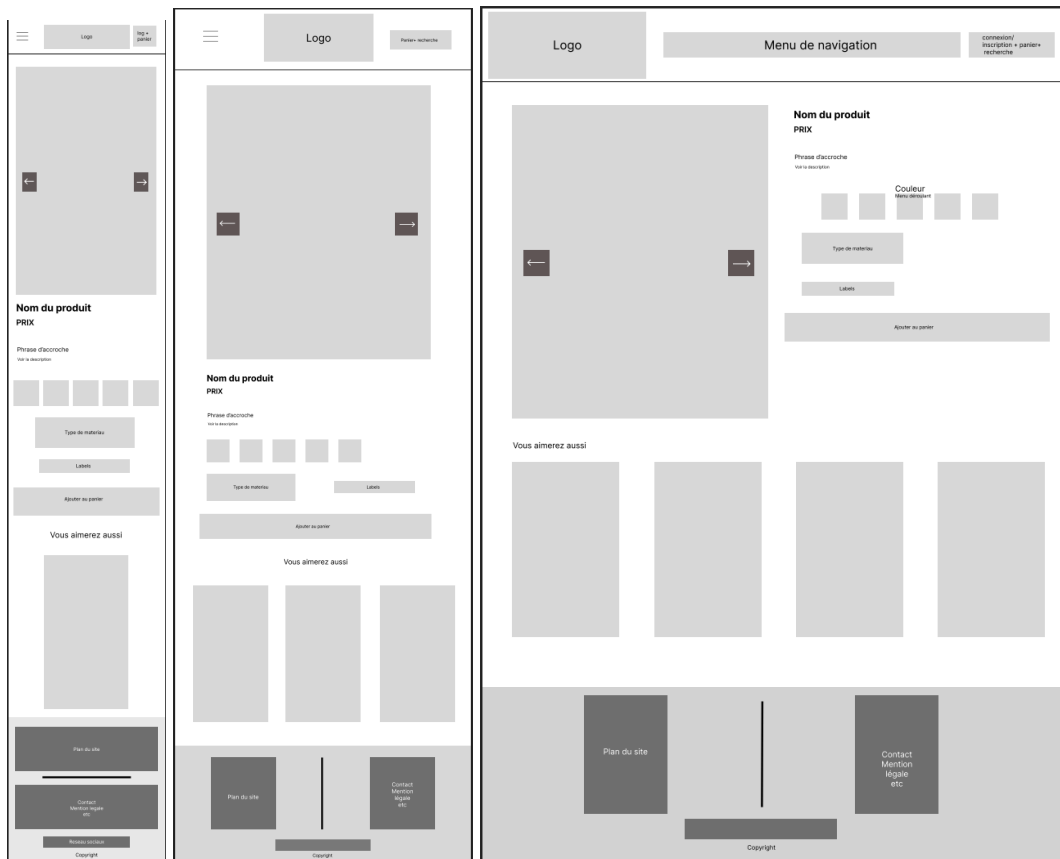
II.c/ Wireframes et maquettes

Comme dit dans la présentation, une personne se formant au marketing digital a rejoint le projet. Elle a donc souhaité reprendre ce que j'avais commencé pour le design du site. C'est pour cela que je n'ai pas toutes les pages de faites, et que je n'ai que des wireframes de produit. Je n'ai pas encore les maquettes du site, celle-ci devant être terminée dans les mois à venir selon la personne. Vous trouverez ci-dessous les wireframes pour la page d'accueil, la liste de produit et la page produit. Certains éléments peuvent être différents car je dois retravailler le design. Ces différences apparaîtront ou non en fonction de si j'ai retravaillé cette partie entre temps.





Page produit



III/ Développement

3.1/ Les étapes du développement

Le développement s'est divisé en deux parties. Le backend a été développé dans un premier temps, et le frontend a été développé dans un second temps. Je vais donc diviser cette partie en deux sous-parties.

3.1.a/ Le backend

Le backend fut la première partie du code développée. La première étape fut de configurer l'ORM pour créer la base de données, ainsi que de lui indiquer où aller chercher les entités. Il vient donc ensuite la création des entités pour créer les tables avec leurs colonnes respectives et les relations entre elles. Suite à ces deux premières étapes, j'ai commencé à m'occuper de tout ce qui touche à l'authentification avec d'abord les routes et services pour créer un compte ainsi que le valider, puis les routes et services liés à la connexion. Les guards pour le compte Admin ainsi que les guards pour les JWT tokens ont été créés dans la foulée.

La troisième étape du backend a été de créer les routes et services du CRUD pour les catégories, sous-catégories, produits, couleurs et matériaux. L'étape suivante était d'intégrer les services bucket S3 d'AWS pour la gestion des images. Cela fait, la suite fut de créer la gestion des stocks pour les créer, les modifier ou les supprimer.

La gestion des produits faites, la suite était d'intégrer les services Stripes pour gérer les paiements, puis d'intégrer les services Mondial Relay ainsi Colissimo pour la gestion des commandes. Il est important de noter que les services Mondial Relay et Colissimo ne sont pas encore complètement implémentés dans le code et demandent encore du travail d'intégration.

La dernière étape était de faire le CRUD pour la gestion des utilisateurs (récupération des informations du profil, modification des informations etc) ainsi que le CRUD des adresses.

3.1.b/ Le frontend

Avant de coder le frontend, certains éléments ont été ajoutés au backend. La gestion des tokens CSRF a été implémentés, puis la protection CORS adaptée pour un projet fullstack.

Dans un premier temps, j'ai créé le header et le footer qui sont des éléments présents sur l'entièreté des pages. Ceci fait, j'ai créé la page d'accueil pour commencer à disposer des éléments et faire les premiers fetch vers l'API NestJs. L'étape suivante fut de créer les formulaires pour les créations de comptes, ainsi que pour la connexion. Pour maintenir cette connexion, un authcontext a été créé. La troisième étape fut la création du dashboard pour permettre à l'administrateur d'effectuer les actions du CRUD via une interface graphique. Ce dashboard lui permet de gérer les stocks, produits et autres. Maintenant qu'il est possible d'ajouter des produits, la création de la page listant les produits et permettant de les filtrer a suivi, avec ensuite la page pour afficher un produit. Pour permettre l'ajout de produit au panier, un cartContext a été implémenté. Celui-ci permet donc d'afficher les produits sur la page panier qui permet alors de rediriger vers le formulaire de création ou de sélection d'adresse. Le chemin menant au paiement, deux pages ont été créées pour être affichées en cas de réussite ou d'échec du paiement. La dernière étape faite au moment où j'écris ces lignes est la page utilisateur qui permet à celui-ci de voir et modifier les informations de son profil, voir ses adresses, ainsi que voir les commandes qu'il a passé.

3.2/ Le code source

Le code est disponible sur ce github:

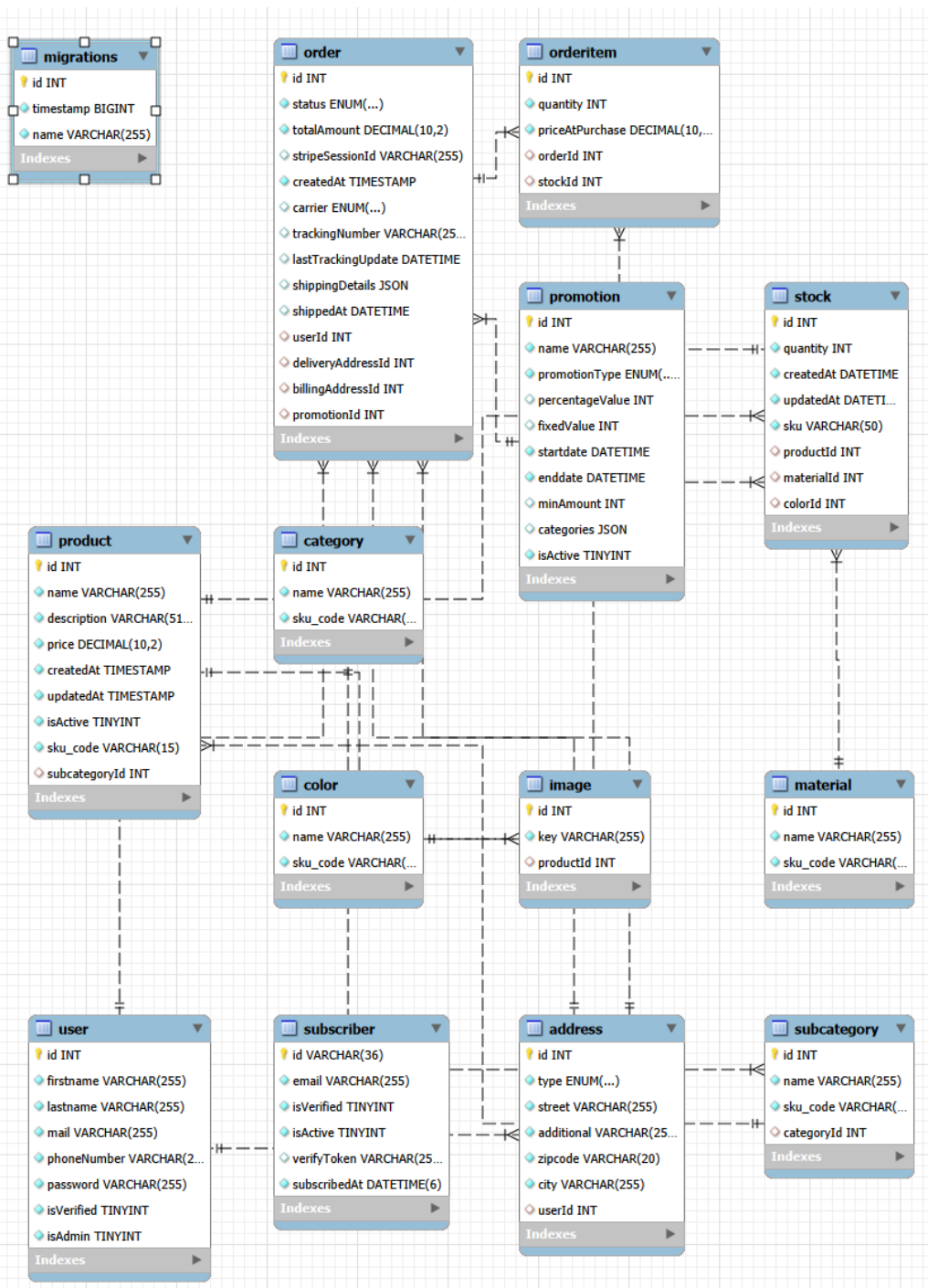
<https://github.com/matiti61290/sac-azzura>

3.2.a/ backend

Dans un souci de simplicité, je ne décrirais que les services de chaque partie étant les fichiers contenant la logique du code. Il sera disponible en annexe 2 les controller, DTO et module de chaque partie présentée.

3.2.a.1/ La base de données

La base de données dont vous pouvez observer le schéma ci-dessous, est composée de 14 tables. La table migrations sert uniquement pour dater les migrations faites par l'ORM qui sera présenté plus tard. La base de données peut se diviser entre deux grandes parties, une partie liée aux produits et une partie liée aux utilisateurs. La partie produit contient tout d'abord les catégories qui sont liées en Many-To-One à la table



“subcategory” (une catégorie peut avoir plusieurs sous-catégories mais l'inverse n'est pas possible). Cette table sous-catégorie est, de la même manière, liée à la table “product”. La

table est alors liée à une table ternaire “stock”, celle-ci traduisant une relation Many-To-Many avec les tables “Color” et “Material”. Ensuite, la partie utilisateur va contenir la table “user” contenant les informations des utilisateurs, mais aussi la table “address” contenant les adresses des utilisateurs et reliée par une relation Many-To-One à la table “user”. Enfin, ces deux groupes de tables sont reliés par la table ternaire order qui relie les tables “user” et “stock” définissant les commandes de chaque utilisateur.

Pour créer la base de données, TypeORM a besoin d'un fichier appelé ormconfig.ts. Ce fichier contient une constante appelée dataSourceOptions. Cette constante contient plusieurs options. Les options type, host, port, username, password et database permettent la connexion à la base de données. L'option bigNumberStrings permet de forcer ou non les objets BIGINT et décimal en string JS. MultipleStatements permet l'envoi de plusieurs requêtes SQL. L'option synchronize permet ou non la création de schéma de la base de données à chaque lancement de l'application. Enfin, les options entières et migrations sont les chemins pour trouver les entités et migrations créées.

```
import { DataSourceOptions, DataSource } from "typeorm";
import * as dotenv from 'dotenv'

dotenv.config()
const isCompiled = __dirname.includes('dist')

export const dataSourceOptions: DataSourceOptions = {
  type: 'mysql',
  host: process.env.DB_HOST,
  port: Number(process.env.DB_PORT) || 3306,
  username: process.env.DB_USER,
  password: process.env.DB_PASSWORD || '',
  database: process.env.DB_DATABASE,
  bigNumberStrings: true,
  multipleStatements: true,
  synchronize: false, //false in prod
  logging: true,
  entities: [ isCompiled ? 'dist/entities/*.entity{.ts,.js}' : 'src/entities/*.entity{.ts,.js}' ],
  migrations: [ isCompiled ? 'dist/database/migrations/*.js,.ts' :
'src/database/migrations/*.ts,.js' ],
  migrationsRun: false
}

export default new DataSource(dataSourceOptions)
```

Les tables de la base de données sont codées sous forme d'entité comme ci-dessous. Cette entité sera prise en exemple pour décrire le code.

Les entités sont créées avec des décorateurs. Le décorateur @Entity indique à l'ORM qu'il s'agit d'une entité. Les décorateurs @Column indiquent les différentes colonnes que contient la table avec le formatage des données (type de données, longueur, nullable etc). Un autre groupe de décorateurs important sont les décorateurs de relation. On en distingue trois: @Many-To-One, @One-To-Many et @Many-To-Many. Le décorateur @Many-To-Many est peu utilisé, souvent remplacé par les décorateurs @Many-To-One ou @One-To-Many reliant les deux tables par une table ternaire.

```
import { Entity, PrimaryGeneratedColumn, Column, ManyToOne, Index, OneToMany } from "typeorm";
import { UserEntity } from "../user.entity";
import { OrderEntity } from "../order.entity";
```

```

import { AddressType } from "../shared/enum/address.enum";
import { Exclude } from "class-transformer";

@Entity('Address')
export class AddressEntity {
    @PrimaryGeneratedColumn({ type: 'int' })
    id!: number

    @Column({ type: "enum", enum: AddressType})
    type!: AddressType

    @Column({ length: 255, nullable: false})
    street!: string

    @Column({ length: 255 })
    additional!: string

    @Column({ length: 20, nullable: false })
    zipcode!: string;

    @Column({ length: 255, nullable: false})
    city!: string;

    @ManyToOne (
        ()=> UserEntity,
        (user)=>user.addresses,
        {onDelete: "CASCADE"}
    )
    @Index()
    @Exclude()
    user!: UserEntity;

    @OneToMany(()=> OrderEntity, (order)=> order.delivery_address)
    orders!: OrderEntity[]
}

```

Un schéma de la base de données est disponible en annexe 1.

3.2.a.2/ L'authentification

La première partie de l'authentification est la création du compte. Plusieurs méthodes pour cette partie sont présentées dans le fichier auth.service.ts. On retrouve d'abord la méthode registration().

```

async registration(registerDto: RegisterDto): Promise<UserEntity> {
    if (registerDto.password !== registerDto.confirmPassword){
        throw new BadRequestException('Les mots de passe ne correspondent pas.')
    }

    const existingUser = await this.userRepository.findOne({ where: {mail: registerDto.mail}})

    if(existingUser){
        throw new ConflictException('Cet email est déjà utilisé.')
    }

    const salt = await bcrypt.genSalt(10)
    const hashedPassword = await bcrypt.hash(registerDto.password, salt)

    const newUser = this.userRepository.create({
        ...registerDto,

```

```

        password: hashedPassword
    })

    await this.userRepository.save(newUser)

    const token = this.jwtService.sign({ id: newUser.id, expiresIn: '1h' })
    await this.confirmMailService.sendVerificationMail(newUser.mail, token)

    return newUser
}

```

Cette méthode va permettre de créer un nouvel utilisateur après avoir vérifié si les mots de passe correspondent, si un utilisateur n'utilise pas déjà cette adresse mail, ainsi qu'avoir crypté le mot de passe. Elle envoie aussi un mail pour demander la validation du compte grâce à la librairie Nodemailer. Les données sont vérifiées grâce à un DTO ou Data Transfer Object comme ci-dessous. C'est ces DTO qui permettent de vérifier si les données reçues sont valides ou non pour les méthodes utilisant ce même DTO.

```

import { IsEmail, IsNotEmpty, IsPhoneNumber, IsString, Matches } from "class-validator";
import { Match } from "../../decorators/password_match.decorator";

export class RegisterDto {
    @IsNotEmpty({ message: 'Le prénom est obligatoire.' })
    @IsString({ message: 'Le prénom doit être une chaîne de caractères.' })
    firstname!: string

    @IsNotEmpty({ message: 'Le nom est obligatoire.' })
    @IsString({ message: 'Le nom doit être une chaîne de caractères.' })
    lastname!: string

    @IsNotEmpty({ message: "L'email est obligatoire." })
    @IsEmail({}, { message: "L'adresse email n'est pas au bon format." }) // <-- La correction est
ici
    mail!: string

    @IsNotEmpty({ message: 'Le numéro de téléphone est obligatoire.' })
    @IsPhoneNumber('FR', { message: 'Le numéro de téléphone n\'est pas valide en France.' })
    phoneNumber!: string

    @IsNotEmpty({ message: 'Le mot de passe est obligatoire.' })
    @IsString()
    @Matches(/^((?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,})$/ , {
        message: 'Le mot de passe doit contenir au moins 8 caractères, une majuscule, un chiffre et
un caractère spécial.'
    })
    password!: string

    @IsNotEmpty({ message: 'La confirmation du mot de passe est obligatoire.' })
    @IsString()
    @Match('password', { message: 'Les mots de passe ne correspondent pas.' })
    confirmPassword!: string
}

```

L'étape suivante est la connexion ainsi que le maintien de celle-ci pour éviter à l'utilisateur de se reconnecter à chaque changement de page. Celle-ci est principalement gérée à l'aide de la librairie passport. Cette librairie permet de gérer plus facilement l'authentification avec NestJs. Celle-ci se divise en trois fichiers, le fichier service contenant les méthodes liées à

l'authentification, le fichier contenant le guard, et le fichier contenant la stratégie. Pour la connexion, on utilise la stratégie locale. Vous trouverez ci-dessous le contrôleur pour la connexion contenant le LocalGuard, les méthodes des services concernant la connexion, ainsi que le guard et la stratégie.

```
@Post('login')
@UseGuards(LocalAuthGuard)
async login(@Req() req: Request, @Res({ passthrough: true }) response: Response) {
  return this.authService.login(req.user, response)
}
```

```
async validateUser(mail: string, password: string): Promise<any> {
  const user = await this.userRepository.findOne({ where: { mail } })
  if(user && (await bcrypt.compare(password, user.password))) {
    const { password, ...result } = user
    return result
  }
  return null
}
```

```
async login(user: any, response: Response) {
  const payload = { mail: user.mail, id: user.id, firstname: user.firstname, isAdmin:
user.isAdmin}
  const token = this.jwtService.sign(payload, { expiresIn: '1h' })

  response.cookie('jwt', token, {
    httpOnly: true,
    secure: false,
    sameSite: 'strict',
    maxAge: 60 * 60 * 1000
  })

  return { message: 'Connexion réussie', user: payload }
}
```

```
import { Injectable } from "@nestjs/common";
import { AuthGuard } from "@nestjs/passport";

@Injectable()
export class LocalAuthGuard extends AuthGuard('local') {}
```

```
import { Injectable } from "@nestjs/common";
import { PassportStrategy } from "@nestjs/passport";
import { Strategy } from "passport-local";
import { AuthService } from "../auth.service";

@Injectable()
export class LocalStrategy extends PassportStrategy(Strategy) {
  constructor(private authService: AuthService) {
    super({ usernameField: 'mail',
      passwordField: 'password'
    })
  }

  async validate(mail: string, password: string): Promise<any> {
    return this.authService.validateUser(mail, password)
  }
}
```

```
}
```

Le fonctionnement de la connexion se fait comme tel. L'utilisateur va entrer ses identifiants. Un DTO va contrôler le format des données dans un premier temps. On va alors rentrer dans le controller sur la route `@Post('login')`. Cette route retourne la fonction login du service. Or, avant de pouvoir rentrer dans le service, le LocalGuard va vérifier les identifiants. Si on regarde le fichier guard, on voit qu'il est défini comme tel:

`AuthGuard('local')`. Ceci indique quelle stratégie suivre. La stratégie locale contient une méthode faisant elle-même appel à la méthode `validateUser()` qui permet de vérifier si l'utilisateur existe bien et s'il a rentré les identifiants corrects. Si tout est bon, le guard laisse le flux continuer pour retourner un token JWT contenant le mail, l'id, le prénom ainsi que si l'utilisateur est admin ou non, et enregistre ce même token dans les cookies.

Maintenant que la connexion est faite, il faut créer une persistance pour que l'utilisateur n'ait pas à se reconnecter à chaque changement de page. Ceci commence avec l'ajout dans les cookies du token JWT. Pour lire ce cookie, la librairie Passport nous donne accès à une autre stratégie, la stratégie JWT comme présentée ci-dessous avec son guard.

```
import { Injectable } from "@nestjs/common";
import { AuthGuard } from "@nestjs/passport";

@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {}
```

```
import { Injectable, NotFoundException } from "@nestjs/common";
import { PassportStrategy } from "@nestjs/passport";
import { Strategy } from "passport-jwt";
import { Request } from "express";
import { UsersService } from "../../user/user.service";

@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor(
    private readonly userService: UsersService
  ) {
    super({
      jwtFromRequest: (request: Request) => {
        return request.cookies?.jwt
      },
      ignoreExpiration: false,
      secretOrKey: process.env.JWT_SECRET as string
    })
  }

  async validate(payload: any) {
    const user = await this.userService.findUserById(payload.id)
    if (!user) {
      throw new NotFoundException
    }

    return user
  }
}
```

Cette fois-ci, le AuthGuard contient le terme `jwt` indiquant qu'il faut utiliser la stratégie JWT. Celle-ci ne va pas contrôler les identifiants de l'utilisateur, mais elle va récupérer le token

JWT dans les cookies, d'où l'importance d'envoyer les cookies lors de call API vers des routes protégées, et vérifier si l'utilisateur existe bien en base de données. Il est important de noter que les tokens JWT sont cryptés et décriptables par une clé définie en amont. Les autres méthodes liées à l'authentification vont être les méthodes pour gérer le changement de mot de passe en cas d'oubli, ainsi que pour renvoyer un mail de vérification. La gestion de l'oubli du mot de passe utilise trois méthodes. La première est une méthode pour envoyer le mail contenant le lien avec le token pour rediriger vers le formulaire. La seconde méthode sert à vérifier le token. Enfin, la dernière méthode sert à changer le mot de passe en base de données. A noter que ces méthodes sont présentées mais seront prochainement adaptées et implémentées dans le frontend.

```
async sendMailForgetPassword(mailDto: MailDto) {
    const payload = {mail: mailDto.mail}
    const token = this.jwtService.sign(payload, {expiresIn: '1h'})

    await this.newPasswordMailService.sendNewPasswordMail(payload.mail, token)
}
```

```
async forgetPassword(token: string) {
    const payload = this.jwtService.verify(token)
    return payload
}

async changePassword(newPassword: NewPasswordDto, token: string) {
    const payload = this.jwtService.verify(token)

    if(!payload.mail){
        throw new NotFoundException
    }

    const user = await this.userRepository.findOne( { where: { mail: payload.mail } })

    if(!user) {
        throw new NotFoundException
    }

    if (newPassword.password !== newPassword.confirmPassword) {
        throw new BadRequestException('Les mots de passe ne correspondent pas')
    }
    const salt = await bcrypt.genSalt(10)
    const hashedPassword = await bcrypt.hash(newPassword.password, salt)

    user.password = hashedPassword

    await this.userRepository.save(user)

    return
}
```

Concernant la méthode pour renvoyer le mail de vérification, celle-ci va vérifier si l'utilisateur est bel-et-bien non vérifié avant de créer un token jwt et utiliser le même mail que pour la vérification pour que l'utilisateur puisse faire cette vérification.

3.2.a.3/ La gestion des produits et des stocks

Comme son nom l'indique, cette partie va contenir le code concernant ce qui touche aux produits et leurs stocks. Cela inclut aussi la gestion des couleurs, matériaux, catégories et sous-catégorie. Pour ces quatre dernières, il ne s'agit que de méthodes pour le CRUD, donc la récupération, la création, la mise à jour et la suppression en base de données de ces éléments. Je ne détaillerai donc pas chaque méthode pour chaque élément, mais je vais utiliser une méthode pour présenter chaque partie du CRUD. Les premières méthodes sont pour récupérer les objets en base de données, ou pour récupérer un élément à l'aide de son ID.

```
async getAllMaterials() {
    const materials = await this.materialRepository.find();

    if (!materials.length) {
        throw new NotFoundException("Matériau non trouvé");
    }

    return materials;
}
```

```
async getMaterialById(materialId: number) {
    const material = await this.materialRepository.findOne({ where: { id: materialId } });

    if (!material) {
        throw new NotFoundException("Matériau non trouvé");
    }

    return material;
}
```

Ensuite, la méthode pour créer un objet va utiliser encore une fois un DTO pour s'assurer du format des données, puis sauvegarder en base de données l'objet.

```
async createMaterial(addMaterialDto: AddMaterialDto) {
    const newMaterial = this.materialRepository.create({ ...addMaterialDto });

    await this.materialRepository.save(newMaterial);

    return newMaterial;
}
```

Le méthode pour mettre à jour un objet est similaire, à la différence que l'on retrouve en paramètre l'id pour retrouver en base de données l'objet à mettre à jour.

```
async updateMaterial(materialId: number, updateMaterialDto: UpdateMaterialDto) {
    const material = await this.materialRepository.findOne({ where: { id: materialId } });

    if (!material) {
        throw new NotFoundException("Matériau non trouvé");
    }

    Object.assign(material, updateMaterialDto);

    return this.materialRepository.save(material);
}
```

Enfin, la dernière méthode est pour supprimer un objet en base de données à partir de son ID.

```
async deleteMaterial(materialId: number) {
    const material = await this.materialRepository.findOne({ where: { id: materialId } });

    if (!material) {
        throw new NotFoundException("Matériau non trouvé");
    }

    return this.materialRepository.remove(material);
}
```

La gestion des produits et des stocks associés est plus complexe. Chaque produit peut avoir des variantes en fonction de la couleur et/ou du matériau utilisé. La gestion des produits induit aussi une gestion des images à part de celle-ci pour enregistrer les images sur le bucket S3 AWS.

Le code ci-dessous est la méthode pour récupérer tous les produits en base de données. Les produits sont récupérés avec les stocks, images, sous-catégorie et catégorie associées. Les images étant enregistrées dans un service tiers, une boucle est créée pour associer les images avec les produits contenant elle-même une boucle pour dans le cas où un produit est associé à plusieurs images.

```
async getAllProducts(): Promise<ProductEntity[]> {
    const products = await this.productRepository.find({
        relations: ["images", "stocks", "subcategory", "subcategory.category"]
    });

    if (!products || products.length === 0) {
        throw new NotFoundException("Aucun produit trouvé");
    }

    return Promise.all(products.map(async (product) => {
        const imageWithUrls = await Promise.all(
            product.images.map(async (image) => {
                const signedUrl = await this.awsS3Service.getFileUrl(image.key);
                return { ...image, url: signedUrl };
            })
        );
        return { ...product, images: imageWithUrls };
    }));
}
```

La méthode pour récupérer un seul produit est dans le même esprit à la différence que l'on utilise l'ID en paramètre pour retrouver le produit, et que les relations récupérées ne sont pas les mêmes.

La grosse partie de ce service est la création d'un produit. Cette méthode va dans un premier temps vérifier si la sous-catégorie sélectionnée pour ce produit existe bien. Elle va ensuite créer un objet produit avec le contenu du formulaire et l'enregistrer. L'étape suivante est d'enregistrer les images correspondantes. Je détaillerai dans une seconde partie le service awsS3Service. Pour le moment, il faut comprendre que la clé des images est formatée pour s'assurer de son unicité, et cette clé est enregistrée en base de données. Il en est de même avec ce qui s'appelle dans la méthode les variations qui sont les différentes variantes du produit avec leur stock associés. Comme pour le awsS3Service, le service stockService sera décrit dans une partie ultérieure. Suite à ces trois étapes, notre produit est

bien enregistré en base de données avec les images et stocks qui lui sont associés. La mise à jour d'un produit est sensiblement le même procédé à la différence que l'on récupère l'objet à l'aide de son ID.

```
async createProduct(
  addProductDto: AddProductDto,
  files: Express.Multer.File[],
  variations: any[]
): Promise<ProductEntity> {

  const subcategoryId = parseInt(addProductDto.subcategoryId.toString(), 10);

  const subcategory = await this.subCategoryRepository.findOne({
    where: { id: subcategoryId }
  });

  if (!subcategory) {
    throw new NotFoundException(`Sous-catégorie introuvable avec l'ID ${subcategoryId}`);
  }

  const product = this.productRepository.create({
    name: addProductDto.name,
    description: addProductDto.description,
    price: parseFloat(addProductDto.price.toString()),
    isActive: true,
    subcategory: subcategory,
    images: [],
    sku_code: addProductDto.sku_code
  });

  const savedProduct = await this.productRepository.save(product);

  const images: ImageEntity[] = [];
  if (files && files.length > 0) {
    for (const file of files) {
      const key = `products/${Date.now()}_${file.originalname}`;
      await this.awsS3Service.uploadFile(file, key);
      const image = this.imageRepository.create({ key, product: savedProduct });
      images.push(image);
    }
    await this.imageRepository.save(images);
  }

  for (const variation of variations) {
    if (variation.quantity > 0) {
      await this.stockService.addStock(
        variation.quantity,
        savedProduct.id,
        parseInt(variation.colorId),
        parseInt(variation.materialId),
        addProductDto.subcategoryId
      );
    }
  }

  return this.findProduct(savedProduct.id);
}
```

Il reste ensuite la suppression d'un produit. Cette méthode utilise l'ID du produit pour le retrouver. Elle va dans un premier temps supprimer les images associées que ce soit leurs

clés dans la base de données, ou les images elle-même enregistrées sur le service tiers. Elle va ensuite supprimer les stocks associés pour enfin pouvoir supprimer le produit sans risque.

```
async deleteProduct(productId: number) {
  const product = await this.productRepository.findOne({
    where: { id: productId },
    relations: ["images", "stocks"]
  });

  if (!product) {
    throw new NotFoundException(`Produit introuvable avec l'ID ${productId}`);
  }

  for (const image of product.images) {
    await this.awsS3Service.deleteFile(image.key);
    await this.imageRepository.remove(image);
  }

  for (const stock of product.stocks) {
    await this.stockService.deleteStockByProductId(stock.sku);
  }

  return this.productRepository.remove(product);
}
```

3.2.a.4/ La gestion des images

Les images ont une gestion particulière. Celles-ci étant enregistrées dans un bucket S3 d'AWS, le CRUD doit être adapté en conséquence.

Le premier fichier est la configuration du bucket S3. Ce petit fichier contient la région du serveur ainsi que les clés d'accès pour notre bucket. Il sert de porte d'entrée pour utiliser les méthodes mis à notre disposition par AWS.

```
import { S3Client } from "@aws-sdk/client-s3";

export const s3Client = new S3Client({
  region: 'eu-north-1',
  credentials: {
    accessKeyId: process.env.AWS_ACCESS_KEY || 'cle',
    secretAccessKey: process.env.AWS_SECRET_KEY || 'cle'
  },
});
```

Le second fichier contient le CRUD pour les images mais aussi pour enregistrer en format PDF les étiquettes pour les expéditions.

La première méthode est la méthode pour enregistrer une image. Celle-ci nous montre comment se construit une commande pour AWS. On indique le nom du bucket, une key qui sera le nom de l'image défini dans la méthode créant les produits, le body qui contient l'image, et enfin le contentType qui contient le format de l'image (JPEG, PNG etc).

```
async uploadFile(file: Express.Multer.File, key: string) {
  const command = new PutObjectCommand({
    Bucket: this.bucketName,
    Key: key,
    Body: file.buffer,
    ContentType: file.mimetype,
  });
}
```

```

    await s3Client.send(command)
    return key
}

```

La seconde méthode est pour récupérer une image. Cette fois, il ne faut que le nom du bucket avec la key pour trouver l'image. On génère ensuite ce qui s'appelle une "signed URL" qui est une URL éphémère qui expirera dans 3600 secondes ici, c'est à dire 1 heure.

```

async getFileUrl(key: string) {
    const command = new GetObjectCommand({
        Bucket: this.bucketName,
        Key: key
    })

    const signedUrl = await getSignedUrl(s3Client, command, { expiresIn: 3600 })
    return signedUrl
}

```

Et enfin, la dernière méthode pour la gestion des images est pour les supprimer. On retrouve encore une fois la même méthodologie que pour les méthodes précédentes.

```

async deleteFile(key: string) {
    const command = new DeleteObjectCommand ({
        Bucket: this.bucketName,
        Key: key
    })
    await s3Client.send(command)
}

```

Comme dit au début de cette partie, une autre méthode est présentée dans ce service, la méthode uploadPdfBuffer(). Celle-ci est identique à la méthode pour ajouter une image dans le bucket S3, à la différence que c'est pour un PDF et non une image.

```

async uploadPdfBuffer(buffer: Buffer, key: string) {
    const command = new PutObjectCommand({
        Bucket: this.bucketName,
        Key: key,
        Body: buffer,
        ContentType: 'application/pdf'
    })

    await s3Client.send(command)
    return key
}

```

3.2.a.5/ La gestion des stocks

La gestion des stocks repose, comme pour les deux parties précédentes, sur le CRUD. Ces stocks sont définis en fonction de la couleur et/ou du matériau. Je vais m'attarder un instant sur la récupération d'un stock spécifique car la méthode change.

```

async getStockBySku(stockSku: string) {
    const stock = await this.stockRepository.findOne({ where: {sku: stockSku}, relations:
["color", "material", "product"]})

    if(!stock){
        throw new NotFoundException
    }
}

```

```

    return stock
}

```

On peut voir que le paramètre pour trouver le stock n'est pas avec un ID, mais avec un code SKU. Un code SKU, ou Stock Keeping Unit, est un code utilisé en logistique pour faciliter la gestion des stocks en créant un code unique à un produit selon des paramètres définis. Je décrirai la création de ce code avec la méthode pour créer un stock. Il existe tout de même une méthode pour récupérer un stock avec son ID qui est plus simple pour certains cas. La méthode pour créer un stock va créer plusieurs relations entre plusieurs tables. On va créer une relation entre un produit, ainsi qu'avec un matériau et une couleur. Pour créer le code SKU, on va aussi récupérer la sous-catégorie du produit, ainsi que sa catégorie. On va alors récupérer le code SKU de chacun pour former le code SKU correspondant au stock. A savoir que le code SKU d'un produit, d'une couleur ou d'une catégorie est un code très court de quelques caractères. On va ensuite associer à ce stock la quantité définie et l'enregistrer en base de données.

```

    async addStock(quantity: number, productId: number, colorId: number, materialId: number,
subcategoryId: number){
        const color = await this.colorRepository.findOne({ where: {id: colorId}})
        const material = await this.materialRepository.findOne({ where: {id: materialId}})
        const product = await this.productRepository.findOne({ where: {id: productId}})
        const subcategory = await this.subcategoryRepository.findOne({ where: {id: subcategoryId},
relations: ["category"]})

        if(!color || !material || !product || !subcategory){
            throw new NotFoundException
        }

        const colorSkuCode = color.sku_code
        const materialSkuCode = material.sku_code
        const productSkuCode = product.sku_code
        const subcategorySkuCode = subcategory.sku_code
        const categorySkuCode = subcategory.category.sku_code

        const stockSkuCode = productSkuCode + "_" + categorySkuCode + "_" + subcategorySkuCode + "_"
+ colorSkuCode + "_" + materialSkuCode

        const stock = this.stockRepository.create({
            quantity: quantity,
            product: product,
            material: material,
            color: color,
            sku: stockSkuCode
        })

        const savedStock = await this.stockRepository.save(stock)

        return this.getStockBySku(savedStock.sku)
    }

```

La méthode pour mettre à jour un stock est un bon exemple d'utilisation du code SKU. On retrouve le stock à l'aide de celui-ci pour mettre à jour la quantité de la variante. On l'utilise aussi pour la suppression d'un stock.

```

    async updateStock(quantity: number, stockSku: string) {
        const stock = await this.stockRepository.findOne({ where: {sku: stockSku}})

```

```

    if(!stock){
        throw new NotFoundException
    }

    stock.quantity = quantity
    const updatedStock = await this.stockRepository.save(stock)

    return this.getStockBySku(updatedStock.sku)
}

```

```

async deleteStockByProductId(stockSku?: string){
    const stock = await this.stockRepository.findOne({ where: {sku: stockSku}})

    if(!stock){
        throw new NotFoundException
    }

    await this.stockRepository.remove(stock)
}

```

3.2.a.6/ Le paiement

La gestion du paiement se divise en trois grosses méthodes principales. Une première méthode pour vérifier la commande et préparer la création de la session de paiement, une seconde méthode pour créer la session de paiement Stripe, et une troisième méthode qui est le webhook pour gérer les stocks en cas de succès du paiement. On retrouve aussi trois autres méthodes pour gérer les échecs de paiement, ou envoyer les mails en cas de succès et d'échec.

La méthode `verificationOrder()` est une méthode qui se divise en trois parties. On a d'abord une première partie qui va récupérer l'utilisateur faisant le paiement, vérifier si son compte a été vérifié, ainsi que récupérer les adresses de livraison et de facturation choisies. Ensuite, on va avoir la préparation des informations du panier tel que son contenu, la valeur totale etc qui vont être formatés pour être enregistrés en base de données ainsi qu'être dans un format utilisable pour Stripe. La dernière partie va être la partie qui va gérer l'utilisation des codes promotionnels en vérifiant si celui-ci est valide, et ensuite déduire sa valeur du prix total.

```

async verificationOrder(cartDto: CartDto, user: UserEntity) {
    const currentUser = await this.userRepository.findOneBy({ id: user.id });

    if (!currentUser) {
        throw new NotFoundException("Utilisateur introuvable");
    }

    if (!currentUser.isVerified) {
        throw new ForbiddenException("Votre compte doit être vérifié pour effectuer un achat.");
    }

    const deliveryAddress = await this.addressRepository.findOneBy({
        id: cartDto.delivery_address_id,
        user: { id: user.id }
    });

    const billingAddress = await this.addressRepository.findOneBy({

```

```

        id: cartDto.billing_address_id,
        user: { id: user.id }
    });

    if (!deliveryAddress || !billingAddress) {
        throw new InternalServerErrorException("Les adresses n'appartiennent pas à cet
utilisateur");
    }

    let totalAmount = 0;

    const order = new OrderEntity();
    order.user = user;
    order.status = OrderStatus.PENDING;
    order.items = [];
    order.delivery_address = deliveryAddress;
    order.billing_address = billingAddress;

    for (const item of cartDto.items) {
        const variant = await this.stockRepository.findOne({ where: { sku: item.sku },
relations: ['product'] });

        if (!variant) {
            throw new NotFoundException("Le produit n'a pas été trouvé");
        }

        if (!variant.product) {
            throw new NotFoundException("Le produit n'a pas été trouvé");
        }

        if (variant.quantity < item.quantity) {
            throw new BadRequestException('Le stock est inférieur à la quantité commandée');
        }

        const orderItem = new OrderItemEntity();
        orderItem.stock = variant;
        orderItem.quantity = item.quantity;
        orderItem.priceAtPurchase = variant.product.price;

        order.items.push(orderItem);

        totalAmount += variant.product.price * item.quantity;
    }

    // Promotion code management ready to be implemented in the frontend
    if (cartDto.promotion_code && cartDto.promotion_code !== "AUCUN" &&
cartDto.promotion_code.trim() !== "") {
        const promotionCode = await this.promotionRepository.findOne({ where: { name:
cartDto.promotion_code } });

        if (!promotionCode) {
            throw new BadRequestException("Le code de promotion renseigné n'existe pas ou a
expiré");
        }

        if (promotionCode.minAmount >= totalAmount) {
            throw new BadRequestException("Vous n'avez pas atteint la valeur minimum pour
utiliser ce code promotionnel")
        } else {
            if (promotionCode.promotionType === "percentage") {

```

```

        const promotionValue = promotionCode.percentageValue;
        totalAmount = totalAmount - (totalAmount * promotionValue) / 100;
    } else if (promotionCode.promotionType === 'fixed_amount') {
        const promotionValue = promotionCode.fixedValue;
        totalAmount = totalAmount - promotionValue;
    }
}

order.totalAmount = totalAmount;

const savedOrder = await this.orderRepository.save(order);

return this.createCheckoutSession(savedOrder.id, currentUser.id, order.items,
currentUser.mail);
}

```

La méthode suivante est une méthode typique pour créer une session de paiement Stripe. Je ne m'attarderai donc pas dessus.

```

async createCheckoutSession(orderId: number, userId: number, items: OrderItemEntity[], userEmail:
string) {
    const line_items = items.map(item => {
        if (!item.stock.product?.name) {
            throw new InternalServerErrorException('Données du produit manquantes')
        }

        return {
            price_data: {
                currency: 'eur',
                product_data: {
                    name: item.stock.product.name,
                    description: `Modèle: ${item.stock.sku}`
                },
                unit_amount: Math.round(item.priceAtPurchase * 100)
            },
            quantity: item.quantity
        }
    });

    try {
        const session = await this.stripe.checkout.sessions.create({
            payment_method_types: ["card"],
            line_items,
            mode: 'payment',
            customer_email: userEmail,
            success_url: 'http://localhost:3000/payment/payment_success',
            cancel_url: `http://localhost:3000/payment/payment_failed/${orderId}/${userId}`,

            metadata: {
                orderId: orderId.toString(),
                userId: userId.toString()
            }
        });

        return { url: session.url };
    } catch (error) {
        console.error("Error creating session: ", error);
        throw new InternalServerErrorException('failed to create checkout session');
    }
}

```

Enfin la dernière partie est le webhook qui fait suite au paiement. Cette méthode va surveiller les événements ayant lieu durant le paiement via la session Stripe. Si un événement indique une complétion de la session de paiement, c'est-à-dire que le processus complet de paiement se termine, le webhook va passer le statut de la commande comme paye, puis mettre à jour les stocks de chaque variante achetée. Il termine en envoyant un mail à l'utilisateur indiquant un succès. A l'inverse, si la session de paiement arrive à expiration, le webhook passe la commande en CANCELLED, donc en commande annulée.

```
async constructEventWebhook(req: any, res: any, signature: string) {
    const endpointSecret = process.env.SECRET_WEBHOOK_KEY;
    if (!endpointSecret) {
        throw new NotFoundException("Le webhook ne fonctionne pas");
    }

    let event: Stripe.Event;
    try {
        event = this.stripe.webhooks.constructEvent(
            req.rawBody,
            signature,
            endpointSecret
        );
    } catch (error) {
        if (error instanceof Error) {
            return res.status(401).send(`webhook error: ${error.message}`);
        } else {
            return res.status(500).send('An unknown error occurred');
        }
    }

    if (event.type === 'checkout.session.completed') {
        const session = event.data.object as Stripe.Checkout.Session;
        const metadata = session.metadata;

        if (!metadata) {
            throw new InternalServerErrorException('Les métadonnées n\'existent pas');
        }

        const orderId = Number(metadata.orderId);
        const userId = Number(metadata.userId);

        try {
            await this.dataSource.transaction(async (transactionalEntityManager) => {
                const order = await transactionalEntityManager.findOne(OrderEntity, {
                    where: { id: orderId },
                    relations: ['items', 'items.stock']
                });

                if (!order || order.status !== OrderStatus.PENDING) {
                    return;
                }

                order.status = OrderStatus.PAID;
                await transactionalEntityManager.save(order);

                for (const line of order.items) {
                    const currentStock = line.stock;

                    if (currentStock.quantity < line.quantity) {
```

```

        throw new InternalServerErrorException(`Stock insuffisant pour l'item
        ${currentStock.sku}`)
    }

    currentStock.quantity -= line.quantity;
    await transactionalEntityManager.save(currentStock);

    console.log(`Transaction réussie. Commande ${orderId} payée et stock
    déduits.`);
    }
    const user = await this.userRepository.findOne({ where: { id: userId } });

    if (!user) {
        throw new InternalServerErrorException(`L'utilisateur avec l'id ${userId}
    n'existe pas`);
    }
    const mail = user.mail;
    await this.sendMailPaymentSuccess(mail, orderId);
    });
    } catch (error) {
        console.error('Échec de la transaction');
    }

    } else if (event.type === 'checkout.session.expired') {
        const session = event.data.object as Stripe.Checkout.Session;
        const metadata = session.metadata;

        if (!metadata) {
            throw new InternalServerErrorException('Les métadonnées n\'existent pas');
        }

        const orderId = Number(metadata.orderId);
        const order = await this.orderRepository.findOne({ where: { id: orderId } });

        if (order && order.status === OrderStatus.PENDING) {
            order.status = OrderStatus.CANCELLED;
            await this.orderRepository.save(order);
        }
    }

    return res.status(200).json({ received: true });
}

```

3.2.a.7/ Gestion des commandes

Je vais séparer en deux parties distinctes la gestion des commandes et la gestion de l'expédition. Je commencerai donc avec la gestion des commandes.

La première chose importante mise en place est une contrainte sur les possibles statuts de commandes. Ceci permet d'éviter que le statut d'une commande expédiée change son statut en "payé" par exemple

```

private readonly allowedTransitions: Record<OrderStatus, OrderStatus[]> = {
    [OrderStatus.PENDING]: [OrderStatus.PAID, OrderStatus.CANCELLED],
    [OrderStatus.PAID]: [OrderStatus.SHIPPED, OrderStatus.CANCELLED],
    [OrderStatus.SHIPPED]: [OrderStatus.DELIVERED],
    [OrderStatus.DELIVERED]: [],
    [OrderStatus.CANCELLED]: []
}

```

Dans un souci de redondance, je ne détaillerai pas à nouveau les méthodes pour récupérer les commandes ou une commande, celle-ci suivant le même principe que les autres méthodes de ce type sans particularité supplémentaire. Il en est de même pour la méthode pour supprimer une commande de la base de données.

Dans le cas des commandes, une méthode a une importance, c'est celle pour mettre à jour le statut d'une commande. Cette commande va contrôler dans un premier temps si le changement de statut est autorisé grâce à notre `allowedTransitions` vu plus haut. Ensuite, si le statut passe en SHIPPED ou expédié en français, le numéro de suivi ainsi que la date d'expédition seront enregistrés en base de données.

```
async updateStatus(orderId: number, updateOrderStatusDto: UpdateOrderStatusDto) {
    const order = await this.orderRepository.findOne({ where: { id: orderId } });

    if (!order) {
        throw new NotFoundException("Aucune commande trouvée");
    }

    const possibleNextStatuses = this.allowedTransitions[order.status];

    if (!possibleNextStatuses || !possibleNextStatuses.includes(updateOrderStatusDto.status)) {
        throw new BadRequestException(`Transition impossible du statut ${order.status} vers le statut ${updateOrderStatusDto.status}`);
    }

    const updateData: any = { status: updateOrderStatusDto.status };

    if (updateOrderStatusDto.status === OrderStatus.SHIPPED) {
        updateData.trackingNumber = updateOrderStatusDto.trackingNumber;
        updateData.shippedAt = new Date();
    }

    await this.orderRepository.update(orderId, updateData);

    return { ...order, ...updateData };
}
```

3.2.a.8/ Gestion des expéditions

Avant de décrire le code dans cette partie, je tiens à signaler que celle-ci n'est pas terminée. Cette partie m'a demandé énormément de temps de travail qui n'est pas encore abouti et risque de me demander encore du temps. L'expédition peut se faire par deux services, Mondial Relay ou Colissimo. Le service colissimo est le plus avancé mais n'a pas encore été intégré au reste du code car je compte retravailler certains morceaux. Quant au service Mondial Relay, il s'agit de mon plus gros point de blocage. Leurs services se basent pour une partie d'entre eux sur le protocole SOAP utilisant le XML. Les framework NodeJs ont du mal avec ce type de protocole qui laisse de plus en plus sa place au protocole REST. J'ai pu échanger avec des techniciens de chacun des services pour leur implémentation mais j'ai préféré laisser cela de côté pour m'assurer de pouvoir rendre quelque chose avant la fin de ma formation d'autant que certaines parties n'ont pas vraiment de moyen d'être testé sans un test grandeur nature.

Le service Colissimo ne contient qu'une seule méthode pour créer l'étiquette d'expédition. Pour tester le service, un payload défini est présent dans la méthode. Ce payload est ensuite envoyé au web service Colissimo. Celui-ci va alors retourner une réponse en multipart de laquelle il faut extraire le contenu de l'étiquette pour obtenir celle-ci en pdf. Le numéro de colis est ensuite enregistré en base de données et l'étiquette est sauvegardée sur le bucket S3. Une méthode supplémentaire est requise pour télécharger via un bouton dans le frontend l'étiquette.

```
public async generateLabel(orderId: number) {

    const order = await this.orderRepository.findOne({ where: {id: orderId}})

    if(!order){
        throw new NotFoundException('L\'Id ne correspond pas a une commande existante.')
    }

    const url = `${this.apiUrl}/generateLabel`

    const payload = {
        outputFormat: {
            outputPrintingType: 'PDF_10x15_300dpi',
            returnType: 'WEB',
        },
        letter: {
            service: {
                productCode: 'DOM',
                depositDate: '2026-04-05', // À dynamiser
            },
            parcel: {
                weight: 1.5,
                nonMachinable: false,
            },
            sender: {
                address: {
                    companyName: 'Ma Boutique',
                    line2: '12 Rue de la Paix',
                    countryCode: 'FR',
                    city: 'Paris',
                    zipCode: '75000',
                },
            },
            addressee: {
                address: {
                    lastName: 'Dupont',
                    firstName: 'Jean',
                    line2: '1 Avenue des Champs-Élysées',
                    countryCode: 'FR',
                    city: 'Paris',
                    zipCode: '75008',
                },
            },
        },
    };

    try{
        const response = await fetch(url, {
            method: 'POST',
            headers: this.getHeaders(),
            body: JSON.stringify(payload)
        })
    }
```

```

    ))

    if(!response.ok){
        const errorText = await response.text()
        this.logger.error(`Erreur HTTP ${response.status}: ${errorText}`)
        throw new Error(`Erreur API Colissimo: ${response.statusText}`)
    }

    const arrayBuffer = await response.arrayBuffer()
    const rawBuffer = Buffer.from(arrayBuffer)

    const contentType = response.headers.get('content-type') || ''
    const boundaryMatch = contentType.match(/boundary="?([^"]+)"?/i)

    if(!boundaryMatch){
        throw new Error("impossible de trouver le boundary de la reponse multipart")
    }
    const boundary = boundaryMatch[1]

    const parts = multipart.parse(rawBuffer, boundary)

    let parcelNumber = null
    let pdfBuffer: Buffer | null = null

    for( const part of parts) {
        if( part.name === 'jsonInfos') {
            const jsonContent = JSON.parse(part.data.toString('utf-8'))
            parcelNumber = jsonContent?.labelV2Response?.parcelNumber
        } else if (part.name === 'label') {
            pdfBuffer = part.data
        }
    }

    if (!pdfBuffer || !parcelNumber) {
        throw new Error("Impossible d'extraire l'étiquette ou le numéro de suivi.")
    }

    const key = `colissimo-shipping-label/${order?.id}-${parcelNumber}.pdf`
    await this.awsS3Service.uploadPdfBuffer(pdfBuffer, key)

    order.trackingNumber = parcelNumber
    order.carrier = Carrier.COLISSIMO
    order.shippedAt = new Date()

    order.shippingDetails = {
        carrier: Carrier.COLISSIMO,
        labelsS3Key: key
    }

    await this.orderRepository.save(order)

    return {
        message: 'Étiquette générée et stockée avec succès !',
        trackingNumber: parcelNumber
    }
} catch (error) {
    this.logger.error('Erreur lors de la generation generateLabel', error)
    throw new InternalServerErrorException('Erreur lors de la creation de l\'etiquette Colissimo')
}
}

```

Le second service, celui de Mondial Relay, est bien plus conséquent. Actuellement, Mondial Relay utilise à la fois une API SOAP et une API REST ayant chacune des fonctions différentes.

La première méthode est une méthode fonctionnant avec l'API SOAP. Elle sert à localiser les points relais et locker. Dans le cas présent, seul le code postal peut être indiqué par le client. Le premier point de blocage que j'ai eu sur cette partie est au niveau de la sécurité de l'API. Comme on le voit, on définit dans un premier temps les arguments à envoyer à l'API, les soapArgs. Ces soapArgs sont utilisés dans une constante appelée securityString. Cette constante est à son tour utilisée pour la méthode de cryptage de l'API. J'ai eu un souci lié à la sécurité de l'API et au protocole SOAP. Ce problème a pu être réglé suite à un appel avec un technicien qui m'a apporté des éléments de réponse me permettant de me débloquent. Suite à cela, j'ai pu récupérer une liste des points relais par rapport au code postal indiqué. La seconde méthode de ce service est la méthode pour créer l'étiquette. Le point de difficulté sur cette partie fut la création du payload qui doit être en XML. Encore une fois, un technicien m'a permis de débloquent le souci et la méthode fonctionne à l'heure actuelle. Les deux dernières méthodes ont la même utilité, mais dans deux contextes différents. Mondial Relay indique que les call vers leur API en boucle peuvent mener à un blocage du compte de l'entreprise. La première méthode va permettre à l'utilisateur de vérifier si le statut a évolué avec une sécurité temporelle pour éviter de faire trop de call. La seconde est un webhook qui va plutôt surveiller si le statut du colis évolue du côté de Mondial Relay.

3.2.a.9/ Gestion du CSRF

Le dernier point du backend que je trouve intéressant est la gestion du CSRF. Celui-ci n'utilise pas un service, mais un middleware directement injecté dans le app.module.ts.

```
import { Injectable, NestMiddleware, UnauthorizedException } from "@nestjs/common";
import { doubleCsrf } from "csrf-csrf";
import { NextFunction, Request, Response } from "express";

const {
  doubleCsrfProtection,
  generateCsrfToken
} = doubleCsrf({
  getSecret: () => "OneKey", //Need to be changed for a secret key in prod
  getSessionIdentifier: (req: Request) => {
    return req.cookies['session-id'] || 'anonymous'
  },
  cookieName: 'x-csrf-token',
  cookieOptions: {
    sameSite: 'none',
    path: '/',
    secure: true
  }
});

@Injectable()
export class CsrfMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    doubleCsrfProtection(req, res, (err) => {
      if(err) {
        throw new UnauthorizedException('Invalid CSRF Token', err.message)
      }
    })
  }
}
```

```

        next()
      })
    }
  }
}

export { generateCsrfToken }

```

Ce middleware est typique d'un middleware pour la sécurité avec un token CSRF. La première partie va être la création des deux méthodes pour générer le token et faire la vérification de celui-ci. La key sera modifiée lors de l'entrée en production du site. Le middleware sert à vérifier si le token est bien présent et valide. La création est assurée dans le controller comme ci-dessous.

```

@Get('token')
getToken(@Req() req: Request, @Res() res: Response) {
  if(!req.cookies['session-id']){
    const sessionId = randomUUID()
    res.cookie('session-id', sessionId, {
      sameSite: 'none',
      path: '/',
      secure: true, //A mettre en true en prod
      httpOnly: true
    })
    req.cookies['session-id'] = sessionId
  }
  const token = generateCsrfToken(req, res)
  return res.json({ csrfToken: token})
}

```

3.2.b/ Le frontend

Cette partie sera plus courte que la précédente, la majorité de la logique étant dans le backend. Par ailleurs, NextJS offre des possibilités qu'il est intéressant de présenter, telles que ses server components et son utilisation de React. La première chose intéressante à présenter est un call API. Le call API ci-dessous est pour envoyer le contenu d'un formulaire. On voit qu'il contient dans son headers le type de contenu, mais aussi le token CSRF car cette route est protégée, mais aussi le token JWT car cette route a une protection JWT.

```

const handleResendMail = async () => {
  setIsSending(true)
  setStatus(null)
  try {
    const csrfToken = await AuthService.getCsrfToken()
    const token = localStorage.getItem('token')

    const res = await fetch(`${process.env.NEXT_PUBLIC_API}auth/resend-verification`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'x-csrf-token': csrfToken,
        ...(token ? { 'Authorization': `Bearer ${token}` } : {})
      },
      credentials: 'include'
    })

    const data = await res.json()
  } catch (error) {
    console.error(error)
  }
}

```

```

    if (!res.ok) {
      throw new Error(data.message || "Impossible de renvoyer le mail pour le moment.")
    }

    setStatus({
      type: 'success',
      message: "Un nouveau lien d'activation vient d'être envoyé sur votre boîte mail ! ✨"
    })
  } catch (err: any) {
    setStatus({
      type: 'error',
      message: err.message
    })
  } finally {
    setIsSending(false)
  }
}

```

React permet aussi la création de fichier Context. Ces types de fichier donnent accès à une partie de code dans l'ensemble du code. C'est par exemple ce qui est utilisé pour la gestion du panier comme ci-dessous. Ce contexte donne accès à plusieurs méthodes dans l'ensemble du code pour ajouter des produits au panier, en retirer, mettre à jour les quantités ainsi que récupérer le contenu pour l'afficher dans le panier

```

'use client'

import { createContext, useContext, useEffect, useState } from 'react';

export interface CartItem {
  stockId: number;
  productId: string | number;
  name: string;
  price: number;
  colorName: string;
  materialName: string;
  imageUrl: string;
  quantity: number;
  sku: string;
}

interface CartContextType {
  cart: CartItem[];
  addToCart: (item: CartItem) => void;
  removeFromCart: (stockId: number) => void;
  updateQuantity: (stockId: number, quantity: number) => void;
  cleanCart: () => void;
  cartTotal: number;
}

const CartContext = createContext<CartContextType | undefined>(undefined);

export function CartProvider({ children }: { children: React.ReactNode }) {
  const [cart, setCart] = useState<CartItem[]>([]);
  const [isLoading, setIsLoaded] = useState(false);

  useEffect(() => {
    const savedCart = localStorage.getItem('sacAzura_cart');
    if (savedCart) {
      setCart(JSON.parse(savedCart));
    }
  });
}

```

```

    }
    setIsLoaded(true);
  }, []);

useEffect(() => {
  if (isLoaded) {
    localStorage.setItem('sacAzura_cart', JSON.stringify(cart));
  }
}, [cart, isLoaded]);

const addToCart = (newItem: CartItem) => {
  setCart((prev) => {
    const existingItem = prev.find((item) => item.stockId === newItem.stockId);
    if (existingItem) {
      return prev.map((item) =>
        item.stockId === newItem.stockId
          ? { ...item, quantity: item.quantity + newItem.quantity }
          : item
      );
    }
    return [...prev, newItem];
  });
};

const removeFromCart = (stockId: number) => {
  setCart((prev) => prev.filter((item) => item.stockId !== stockId));
};

const updateQuantity = (stockId: number, quantity: number) => {
  if (quantity < 1) return;
  setCart((prev) =>
    prev.map((item) => (item.stockId === stockId ? { ...item, quantity } : item))
  );
};

const cleanCart = () => {
  if (typeof window !== 'undefined') {
    localStorage.removeItem('sacAzura_cart')
  }
  setCart([])
}

const cartTotal = cart.reduce((total, item) => total + item.price * item.quantity, 0);

return (
  <CartContext.Provider value={{ cart, addToCart, removeFromCart, updateQuantity, cleanCart,
cartTotal }}>
    {children}
  </CartContext.Provider>
);
}

export function useCart() {
  const context = useContext(CartContext);
  if (context === undefined) {
    throw new Error('useCart doit être utilisé dans un CartProvider');
  }
  return context;
}

```

IV/ Test et validation

Au cours de ce projet, les trois types de test ont été utilisés. Je vais succinctement présenter comment chacun des types ont été utilisés, puis comment chacun a pu m'aider à mener une correction a mon code.

4.1/ Test unitaire

Les tests unitaire ont été réalisés à l'aide de la librairie Jest. Un test unitaire avec celui-ci peut se séparer en deux parties. On a d'abord la partie qui va mocker, ou reproduire, ce dont les tests auront besoins tel que les repositories, les services etc.

```
jest.mock('bcrypt');

describe('AuthService', () => {
  let authService: AuthService;
  let userRepository: any;
  let jwtService: any;
  let confirmMailService: any;
  let mockNewPasswordMailServiceInstance: any;

  const mockUserRepository = {
    findOne: jest.fn(),
    create: jest.fn(),
    save: jest.fn(),
  };

  const mockJwtService = {
    sign: jest.fn(),
    verify: jest.fn(),
  };

  const mockConfirmMailService = {
    sendVerificationMail: jest.fn(),
  };

  const mockNewPasswordMailService = {
    sendNewPasswordMail: jest.fn(),
  };

  beforeEach(async () => {
    const module: TestingModule = await Test.createTestingModule({
      providers: [
        AuthService,
        {
          provide: getRepositoryToken(UserEntity),
          useValue: mockUserRepository,
        },
        {
          provide: JwtService,
          useValue: mockJwtService,
        },
        {
          provide: ConfirmMailService,
          useValue: mockConfirmMailService,
        },
        {
          provide: newPasswordMailService,
        },
      ],
    }).compile();

    authService = module.get(AuthService);
    userRepository = module.get(getRepositoryToken(UserEntity));
    jwtService = module.get(JwtService);
    confirmMailService = module.get(ConfirmMailService);
    mockNewPasswordMailServiceInstance = module.get(newPasswordMailService);
  });
});
```

```

        useValue: mockNewPasswordMailService,
    },
],
}).compile();

authService = module.get<AuthService>(AuthService);
userRepository = module.get(getRepositoryToken(UserEntity));
jwtService = module.get<JwtService>(JwtService);
confirmMailService = module.get<ConfirmMailService>(ConfirmMailService);
mockNewPasswordMailServiceInstance = module.get<newPasswordMailService>(newPasswordMailService);
});

afterEach(() => {
    jest.clearAllMocks();
});

```

Ensuite, on retrouve tous les tests pour tester chaque méthode du service testé comme montré dans le test ci-dessous. Pour être honnête, ce type de test ne m'a pas énormément apporté lors du développement car j'ai principalement utilisé les tests fonctionnels pour le développement. Par ailleurs, ce type de test qui, pour la grosse majorité ont directement été validé avec succès, m'ont permis de remarquer des petites erreurs de code qui pouvaient s'avérer problématique dans certains cas de figure. On retrouve ce cas de figure avec les images ou une erreur ressortait car dans certaines situations, il n'y a pas encore d'url défini pour une image, ou celle-ci a pu être supprimée directement dans le bucket S3. C'est pour cela que l'on retrouve dans l'entité ImageEntity la présence d'une colonne virtuelle "url" qui n'est pas prise en compte lors de la création de la base de données.

```

describe('registration', () => {
    const registerDto: RegisterDto = {
        firstname: 'Jean',
        lastname: 'Dupont',
        mail: 'jean@example.com',
        phoneNumber: '+33612345678',
        password: 'Password123!',
        confirmPassword: 'Password123!',
    };

    it('devrait jeter une BadRequestException si les mots de passe ne correspondent pas', async () => {
        const badDto = { ...registerDto, confirmPassword: 'differentPassword' };
        await expect(authService.registration(badDto)).rejects.toThrow(BadRequestException);
    });

    it('devrait jeter une ConflictException si le mail est déjà pris', async () => {
        userRepository.findOne.mockResolvedValue({ id: 1, mail: 'jean@example.com' });
        await expect(authService.registration(registerDto)).rejects.toThrow(ConflictException);
    });

    it('devrait créer et retourner un utilisateur avec succès', async () => {
        userRepository.findOne.mockResolvedValue(null);
        (bcrypt.genSalt as jest.Mock).mockResolvedValue('salt');
        (bcrypt.hash as jest.Mock).mockResolvedValue('hashed_password');

        const mockCreatedUser = { id: 123, ...registerDto, password: 'hashed_password' };
        userRepository.create.mockReturnValue(mockCreatedUser);
        userRepository.save.mockResolvedValue(mockCreatedUser);
        jwtService.sign.mockReturnValue('verification-token');
    });

```

```

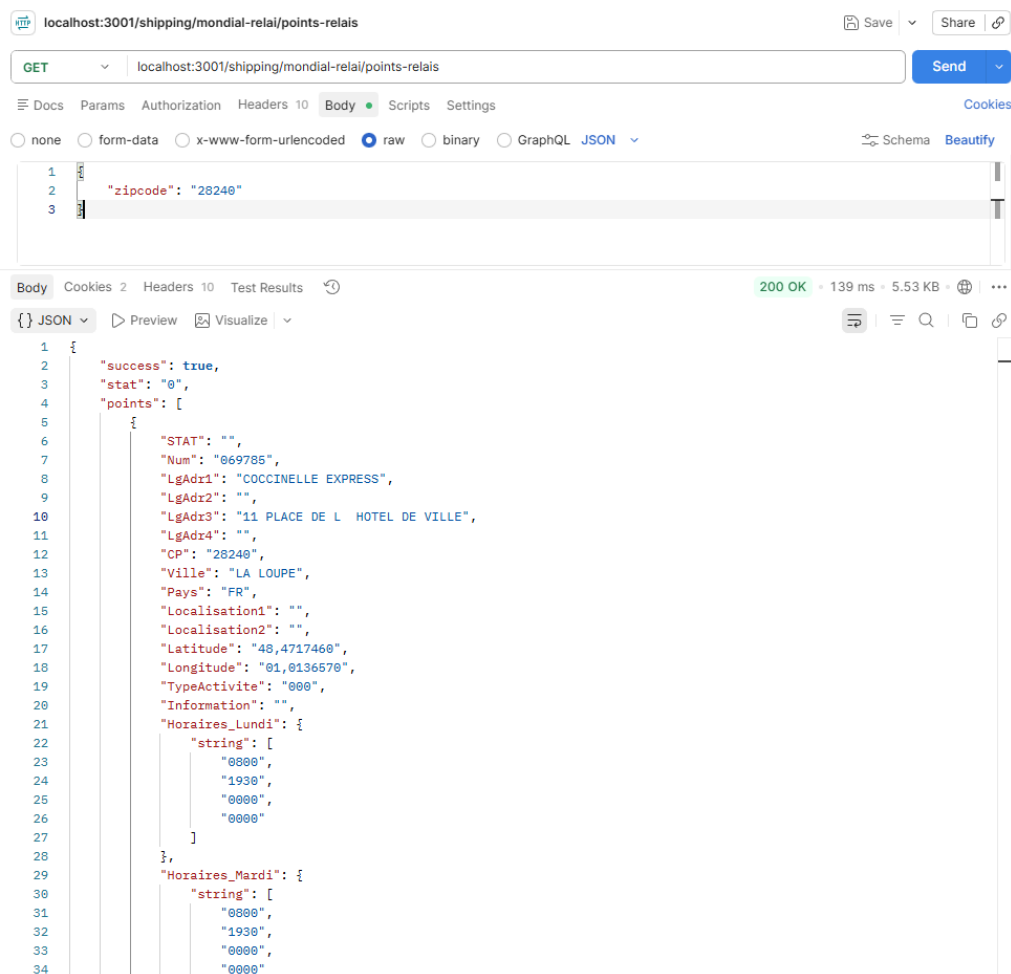
const result = await authService.registration(registerDto);

expect(userRepository.findOne).toHaveBeenCalledWith({ where: { mail: registerDto.mail } });
expect(userRepository.create).toHaveBeenCalledWith({ ...registerDto, password:
'hashed_password' });
expect(userRepository.save).toHaveBeenCalledWith(mockCreatedUser);
expect(confirmMailService.sendVerificationMail).toHaveBeenCalledWith('jean@example.com',
'verification-token');
expect(result).toEqual(mockCreatedUser);
});
});

```

4.2/ Test fonctionnel

Ce type de test a été l'élément central dans le développement du backend. J'ai utilisé pour cela le logiciel Postman. Ces tests consistaient à tester les routes du site. Cela permet de tester le controller avec le service. Dans l'exemple ci-dessous, on voit que j'appelle la route pour récupérer les points-relais. On indique le code postal en format string comme demandé par le service Mondial Relay. La route nous retourne bien une liste des points-relais dans le département. Ce type de test m'a permis de corriger énormément d'erreurs dues à des mauvaises urls ou encore des problèmes de format de données. Ils m'auront aussi fortement aider dans la gestion des erreurs.



4.3/ Test d'intégration

Les tests d'intégration se sont fait principalement avec la création du frontend. Ceci ont principalement consisté en la vérification des call API et de ce qu'ils retournent. Le code étant nettoyé des console.log, je n'ai pas d'exemple à montrer sous la main. On peut voir que ces tests ont bien fonctionné car les call API retournent bien ce qu'elles sont censées retourner.

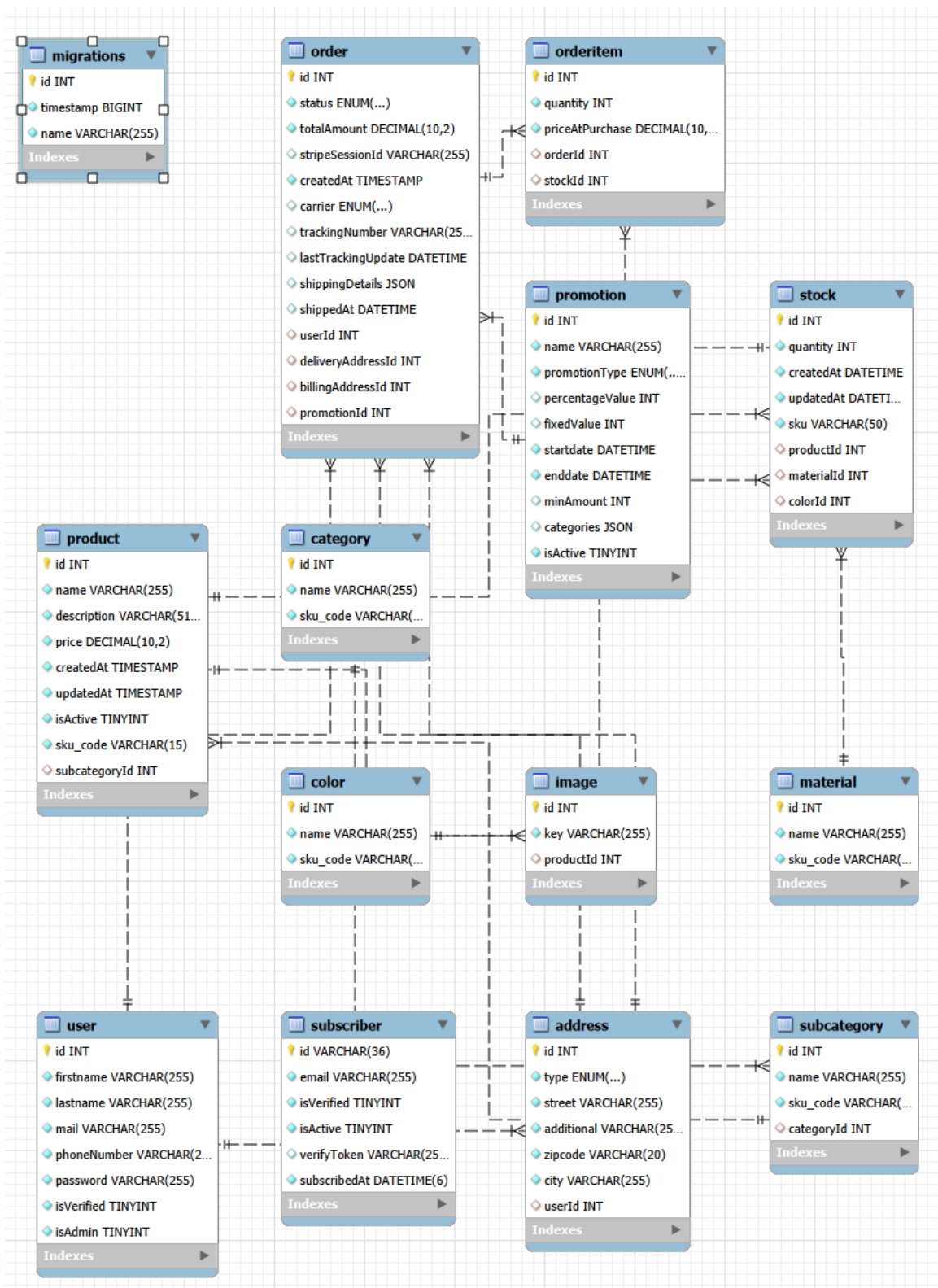
VI/ Bilan personnel

Ce projet m'a énormément apporté en termes de compétences de développement. Malgré mon sentiment personnel d'avoir potentiellement visé un projet de trop grosse taille pour un premier projet, j'ai pu apprendre comment fonctionne une application fullstack, mais aussi tout ce qui touche à la sécurité sur ce type d'application. Ce projet aura aussi été pour moi l'occasion de tester certaines utilisations de l'IA en me basant sur des retours d'expérience de plusieurs développeurs senior. J'ai par exemple pu tester un agent IA local que je n'ai utilisé que pour créer les templates de base pour le front que j'ai pu retravailler ensuite, ou pour créer des bases de codes simples comme des call API basiques. Ce projet m'aura aussi apporté de l'expérience sur le travail avec un client et une équipe en dehors du développement étant donné qu'une personne s'occupe de faire le design et de préparer le SEO, le tout en suivant les demandes d'un client.

Par ailleurs, je pense que je manque encore d'un élément très important dans le développement, le travail en équipe. La formation suivie en plus des conseils apportés par mes rencontres m'ont apporté une approche théorique que je n'ai encore jamais pu expérimenter.

Le gros point fort de ce site est son potentiel en termes de compétence que je peux développer. Ce type de projet demande plusieurs compétences et l'utilisation ensemble de plusieurs services tiers (AWS, Colissimo, Stripe etc). Je découvre aussi plusieurs choses sur la logistique et comment l'intégrer au sein d'une boutique en ligne. Par ailleurs, comme dit plus haut, les deux plus gros points faibles est que je n'ai pas pu développer de compétence pour ce qui est de travailler sur un projet en équipe, et le projet est un très gros projet pour une seule personne en sortant de formation.

Annexe



Annexe 2: Codes supplémentaires

Authentication

```
import { Module } from "@nestjs/common";
import { AuthController } from "../auth.controller";
import { AuthService } from "../auth.service";
import { TypeOrmModule } from "@nestjs/typeorm";

import { JwtModule } from "@nestjs/jwt";
import { ConfirmMailService } from "../authMail/confirmMail.service";
import * as dotenv from 'dotenv'
import { PassportModule } from "@nestjs/passport";
import { LocalStrategy } from "../strategies/local.strategy"
import { JwtStrategy } from "../strategies/jwt.strategy"
import { JwtAuthGuard } from "../guards/jwt-auth.guard";
import { newPasswordMailService } from "../authMail/newPasswordMail.service";
import { UsersService } from "../user/user.service";
import { AdminGuard } from "../guards/admin.guard";
import { UserEntity } from "../entities/user.entity";

dotenv.config()

@Module({
  imports:[TypeOrmModule.forFeature([UserEntity]),
    JwtModule.register({
      secret: process.env.JWT_SECRET,
      signOptions: {expiresIn: '60s'}
    }),
    PassportModule.register({ defaultStrategy: 'local'})],
  controllers: [AuthController, ],
  providers: [AuthService, UsersService, ConfirmMailService, LocalStrategy, JwtStrategy,
JwtAuthGuard, newPasswordMailService, AdminGuard],
  exports:[AuthService, JwtModule, AdminGuard]
})

export class AuthModule {}
```

```
import { Body, Controller, Get, Post, Query, Req, Res, UseGuards, UsePipes, ValidationPipe } from
"@nestjs/common";
import { AuthService } from "../auth.service";
import { LocalAuthGuard } from "../guards/local-auth.guard";
import { JwtAuthGuard } from "../guards/jwt-auth.guard";
import type { Request, Response, } from "express";
import { RegisterDto } from "../shared/dtos/auth/register.dto";
import { MailDto } from "../shared/dtos/auth/mail.dtos";
import { NewPasswordDto } from "../shared/dtos/auth/newPassword.dto";

@Controller('auth')
export class AuthController {
  constructor(
    private readonly authService: AuthService,
  ){}

  @Post('register')
  @UsePipes( new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true}))
  async registrationUser(@Body() registerDto: RegisterDto){
    return this.authService.registration(registerDto)
  }

  @Get('validation-user')
  async validationUser(@Query('token') token: string){
```

```

        await this.authService.validateAccount(token)
        return { message: 'Utilisateur valide' }
    }

    @Post('login')
    @UseGuards(LocalAuthGuard)
    async login(@Req() req: Request, @Res({ passthrough: true }) response: Response) {
        return this.authService.login(req.user, response)
    }

    @Get('profile')
    @UseGuards(JwtAuthGuard)
    async getProfile(@Req() req: Request) {
        return req.user
    }

    @Post('mail-forget-password')
    async mailForgetPassword(@Body() mailDto: MailDto) {
        return this.authService.sendMailForgetPassword(mailDto)
    }

    //Not yet implemented
    @Get('forget-password')
    async forgetPassword(@Query('token') token: string) {
        return this.authService.forgetPassword(token)
    }

    //Not yet implemented
    @Post('change-password')
    async changePassword(@Query('token') token: string,
        @Body() newPasswordDto: NewPasswordDto) {
        return this.authService.changePassword(newPasswordDto, token)
    }

    @Post('resend-verification')
    @UseGuards(JwtAuthGuard)
    async resendVerification(@Req() req: Request) {
        return this.authService.resendVerification(req.user);
    }
}

```

```

import { Injectable } from "@nestjs/common";
import * as nodemailer from 'nodemailer'

@Injectable()
export class ConfirmMailService {
    private transporter = nodemailer.createTransport({
        service: 'gmail',
        auth: {
            user: process.env.USER_MAIL,
            pass: process.env.PASSWORD_MAIL
        }
    })
}

async sendVerificationMail(mail: string, token: string) {
    const link = `http://localhost:3001/auth/validation-user?token=${token}`
    await this.transporter.sendMail({
        from: 'barbeymathieudev@gmail.com',
        to: mail,
        subject: 'Validation de votre compte',
    })
}

```

```

        text: `Cliquez ici pour valider l'activation du compte : ${link}`,
        html: `
            <div>
                <h1>Merci de votre inscription</h1>
                <h3>Valider des maintenant votre compte!</h3>
                <p>Cliquez sur ce <a href="${link}">lien</a> pour valider votre compte.</p>
            </div>
        `
    })
}
}

```

```

import { Injectable } from "@nestjs/common";
import * as nodemailer from 'nodemailer'

@Injectable()
export class newPasswordMailService {
    private transporter = nodemailer.createTransport({
        service: 'gmail',
        auth:{
            user: process.env.USER_MAIL,
            pass: process.env.PASSWORD_MAIL
        }
    })

    async sendNewPasswordMail(mail:string, token: string) {
        const link = `http://localhost:3001/auth/forget-password?token=${token}`
        await this.transporter.sendMail({
            from: 'barbeymathieudev@gmail.com',
            to: mail,
            subject: 'Changement de mot de passe',
            text: `Cliquez sur ce lien pour modifier votre mot de passe: ${link}`,
            html: `
                <div>
                    <h3>Changer votre mot de passe en cliquant sur le lien ci-dessous:</h3>
                    <a href="${link}">${link}</a>
                </div>
            `
        })
    }
}

```

Produit et stock

```

import { Body, Controller, Delete, Get, Param, ParseIntPipe, Patch,
Post, Query, UploadedFiles, UseGuards, UseInterceptors } from
"@nestjs/common";
import { ProductService } from "../product.service";
import { JwtAuthGuard } from "../auth/guards/jwt-auth.guard";
import { AdminGuard } from "../auth/guards/admin.guard";
import { FilesInterceptor } from "@nestjs/platform-express";
import { AddProductDto } from
"../../shared/dtos/product/addProduct.dto";
import { UpdateProductDto } from
"../../shared/dtos/product/updateProduct.dto";
@Controller('products')

```

```

export class ProductController {
  constructor(
    private readonly productService: ProductService
  ) {}

  @Get('')
  async getAllProduct() {
    return this.productService.getAllProducts()
  }

  @Get('/:productId')
  async findProduct(
    @Param('productId', ParseIntPipe) productId: number
  ) {
    let product = await this.productService.findProduct(productId)

    return product
  }

  @Post('add-product')
  @UseGuards(JwtAuthGuard, AdminGuard)
  @UseInterceptors(FilesInterceptor('files'))
  async addProduct(
    @Body() addProductDto: AddProductDto,
    @UploadedFiles() files: Express.Multer.File[]
  ) {
    // string turned into JS table
    const variationsArray = JSON.parse(addProductDto.variations);

    return this.productService.createProduct(addProductDto, files, variationsArray);
  }

  @Patch('update-product/:productId')
  @UseGuards(JwtAuthGuard, AdminGuard)
  @UseInterceptors(FilesInterceptor('files'))
  async updateProduct(
    @Param('productId', ParseIntPipe) productId: number,
    @Body() updateProductDto: UpdateProductDto,
    @UploadedFiles() files: Express.Multer.File[]
  ) {
    updateProductDto.files = files
    return this.productService.updateProduct(productId, updateProductDto)
  }

  @Delete('delete-product/:productId')
  @UseGuards(JwtAuthGuard, AdminGuard)
  async deleteProduct(
    @Param('productId', ParseIntPipe) productId: number
  ) {
    this.productService.deleteProduct(productId)
    return "Produit supprimé"
  }
}

```

```

import { Controller, Delete, Get, Param, ParseIntPipe, UseGuards } from "@nestjs/common";
import { StockService } from "../stock.service";
import { JwtAuthGuard } from "../../auth/guards/jwt-auth.guard";
import { AdminGuard } from "../../auth/guards/admin.guard";

@Controller('stocks')

```

```

export class StockController {
  constructor(
    private readonly stockService: StockService
  ) {}

  @Get('')
  @UseGuards(JwtAuthGuard, AdminGuard)
  async getAllStock() {
    return this.stockService.getAllStock()
  }

  @Delete('delete-stock/:stockId')
  @UseGuards(JwtAuthGuard, AdminGuard)
  async deleteStock(
    @Param('stockId', ParseIntPipe) stockId: number
  ) {
    const stock = await this.stockService.getStockById(stockId)
    const stockSku = stock.sku
    this.stockService.deleteStockByProductId(stockSku)
    return "Stock supprime"
  }
}

```

Paiement

```

import { Body, Controller, Get, Param, Headers, HttpStatusCode, HttpStatus, ParseIntPipe, Post, Req, Res,
  UseGuards } from "@nestjs/common";
import { PaymentService } from "../payment.service";
import type { Request, Response } from "express";
import { JwtAuthGuard } from "../../auth/guards/jwt-auth.guard";
import { CartDto } from "../../../shared/dtos/payment/cart.dto";

@Controller('payment')
export class PaymentController {

  constructor(
    private readonly paymentService: PaymentService
  ) {}

  @UseGuards(JwtAuthGuard)
  @Post('')
  async checkoutSession(
    @Body() cartDto: CartDto,
    @Req() req
  ) {
    const user = req.user
    console.log("L'id de l'user est:", user)
    return this.paymentService.verificationOrder(cartDto, user)
  }

  //Rework planned with frontend
  @Get('payment_failed/:orderId/:userId')
  async paymentFailed(
    @Param('orderId', ParseIntPipe) orderId: number,
    @Param('userId', ParseIntPipe) userId: number
  ) {
    await this.paymentService.paymentFailed(orderId, userId)

    return "la commande a echoué"
  }
}

```

```

@Post('webhook')
@HttpCode(HttpStatus.OK)
async handleStripeWebhook(
  @Req() req: Request,
  @Res() res: Response,
  @Headers('stripe-signature') signature: string
){
  return this.paymentService.constructEventWebhook(req, res, signature)
}
}

```

```

import { Injectable } from "@nestjs/common";
import { InjectRepository } from "@nestjs/typeorm";
import * as nodemailer from 'nodemailer'
import { Repository } from "typeorm";
import { OrderEntity } from "../../entities/order.entity";

@Injectable()
export class PaymentSuccessMailService {

  constructor(
    @InjectRepository(OrderEntity)
    private readonly orderRepository: Repository<OrderEntity>
  ){}

  private transporter = nodemailer.createTransport({
    service: 'gmail',
    auth: {
      user: process.env.USER_MAIL,
      pass: process.env.PASSWORD_MAIL
    }
  })

  async sendPaymentSuccessMail(mail: string, orderId: number) {
    const order = await this.orderRepository.findOne({ where: {id: orderId}})

    await this.transporter.sendMail({
      from: 'barbeymathieudev@gmail.com',
      to: mail,
      subject: 'Confirmation de paiement',
      text: 'Votre paiement a été validé',
      html: `
        <div>
          <h3>Votre paiement pour la commande ${orderId} a été validé</h3>
          <p>Liste de la commande a implementer</p>
        </div>
      `
    })
  }
}

```

Gestion des commandes

```

import { Body, Controller, Delete, Get, Param, ParseIntPipe, Patch, Post, Req, UseGuards } from
"@nestjs/common";
import { OrderService } from "../order.service";
import { JwtAuthGuard } from "../../auth/guards/jwt-auth.guard";
import { AdminGuard } from "../../auth/guards/admin.guard";
import { UpdateOrderStatusDto } from "../../shared/dtos/order/updateOrderStatus.dto";

@Controller('orders')

```

```

export class OrderController {
  constructor(
    private readonly orderService: OrderService
  ) {}

  @Get('')
  @UseGuards(JwtAuthGuard, AdminGuard)
  async getAllOrder() {
    return this.orderService.getAllOrder()
  }

  @Get('user/orders')
  @UseGuards(JwtAuthGuard)
  async getOrdersByUser(
    @Req() req
  ){
    const user = req.user
    return this.orderService.getOrdersByUser(user)
  }

  @Get('/:orderId')
  @UseGuards(JwtAuthGuard, AdminGuard)
  async getOrderById(
    @Param('orderId', ParseIntPipe) orderId: number
  ) {
    return this.orderService.getOrderById(orderId)
  }

  @Get("/user/orders/:orderId")
  @UseGuards(JwtAuthGuard)
  async GetOrderByIdByUser(
    @Req() req,
    @Param('orderId', ParseIntPipe) orderId: number
  ){
    const user = req.user
    return this.orderService.getOrderByIdByUser(user, orderId)
  }

  @Patch("/:orderId")
  @UseGuards(JwtAuthGuard, AdminGuard)
  async updateStatus(
    @Param('orderId', ParseIntPipe) orderId: number,
    @Body() updateOrderStatusDto: UpdateOrderStatusDto
  ){
    return this.orderService.updateStatus(orderId, updateOrderStatusDto)
  }

  @Delete("delete-order/:orderId")
  @UseGuards(JwtAuthGuard, AdminGuard)
  async deleteMaterial(
    @Param('orderId', ParseIntPipe) orderId: number
  ) {
    return this.orderService.deleteOrder(orderId)
  }
}

```

Gestion des expéditions

```
import { Controller, Get, Param, Post, Body, ParseIntPipe, HttpStatusCode, HttpStatus, Query, UnauthorizedException } from '@nestjs/common';
import { MondialRelayService } from '../service/mondial-relai.service';
import { ColissimoService } from '../service/colissimo.service';
import { FindRelayPointDto } from '../../shared/dtos/mondial_relai/findRelayPoint.dto';
import { CreateLabelDto } from '../../shared/dtos/mondial_relai/createLabelDto.dto';

/*
Shipping parts don't work for the moment
*/
@Controller('shipping')
export class ShippingController {
  constructor(
    private readonly mondialRelayService: MondialRelayService,
    private readonly colissimoService: ColissimoService,
  ) {}

  //Mondial Relay
  @Get('mondial-relai/points-relais')
  async getPointsRelais(
    @Body() findRelayPointDto: FindRelayPointDto
  ) {
    return await this.mondialRelayService.rechercherPointsRelais(findRelayPointDto);
  }

  @Post('mondial-relai/create-label')
  async generateMrLabel(
    @Body() createLabelDto: CreateLabelDto
  ) {
    return await this.mondialRelayService.createLabel(createLabelDto);
  }

  @Get('mondial-relai/tracing-package/:orderId')
  async tracingPackage(
    @Param('orderId', ParseIntPipe) orderId: number
  ) {
    return this.mondialRelayService.tracingPackage(orderId);
  }

  @Post('mondial-relay/webhook')
  @HttpStatusCode(HttpStatus.OK)
  async handleMondialRelayWebhook(
    @Query('token') token: string,
    @Body() payload: any
  ) {
    await this.mondialRelayService.handleWebhook(payload, token);
  }

  //Colissimo
  @Post('colissimo/generate-label/:orderId')
  async generateColissimoLabel(
    @Param('orderId', ParseIntPipe) orderId: number
  ) {
    return this.colissimoService.generateLabel(orderId);
  }
}
```

CSRF

```
import { Controller, Get, Req, Res } from "@nestjs/common";
import { randomUUID } from "crypto";
import type { Request, Response } from "express";
import { generateCsrfToken } from "../csrf.middleware";

@Controller('csrf')
export class CsrfController {
  @Get('token')
  getToken(@Req() req: Request, @Res() res: Response) {
    if(!req.cookies['session-id']){
      const sessionId = randomUUID()
      res.cookie('session-id', sessionId, {
        sameSite: 'none',
        path: '/',
        secure: true, //A mettre en true en prod
        httpOnly: true
      })
      req.cookies['session-id'] = sessionId
    }
    const token = generateCsrfToken(req, res)
    return res.json({ csrfToken: token})
  }
}
```

Annexe 3: Tests

Exemple de test unitaire avec le test du paiement

```
import { Test, TestingModule } from '@nestjs/testing';
import { PaymentService } from '../payment.service';
import { getRepositoryToken } from '@nestjs/typeorm';
import { BadRequestException, ForbiddenException, InternalServerErrorException, NotFoundException }
from '@nestjs/common';
import { DataSource } from 'typeorm';
import { StockEntity } from '../../entities/stock.entity';
import { OrderEntity } from '../../entities/order.entity';
import { UserEntity } from '../../entities/user.entity';
import { AddressEntity } from '../../entities/addresses.entity';
import { PromotionEntity } from '../../entities/promotion.entity';
import { PaymentSuccessMailService } from '../paymentMail/paymentSuccessMail.service';
import { PaymentFailMailService } from '../paymentMail/paymentFailMail.service';
import { CartDto } from '../../shared/dtos/payment/cart.dto';
import { OrderStatus } from '../../shared/enum/order.enum';

jest.mock('stripe', () => {
  return {
    Stripe: jest.fn().mockImplementation(() => ({
      checkout: {
        sessions: {
          create: jest.fn(),
        },
      },
      webhooks: {
        constructEvent: jest.fn(),
      },
    })),
  };
});
```

```

describe('PaymentService', () => {
  let paymentService: PaymentService;
  let stockRepository: any;
  let orderRepository: any;
  let userRepository: any;
  let addressRepository: any;
  let promotionRepository: any;
  let dataSource: any;
  let mockPaymentSuccessMail: any;
  let mockPaymentFailMail: any;

  const originalEnv = process.env;

  beforeEach(async () => {
    process.env = { ...originalEnv };
    process.env.SECRET_KEY_STRIPE = 'sk_test_mocked_key';
    process.env.SECRET_WEBHOOK_KEY = 'whsec_mocked_key';

    const module: TestingModule = await Test.createTestingModule({
      providers: [
        PaymentService,
        { provide: getRepositoryToken(StockEntity), useValue: { findOne: jest.fn(), save: jest.fn() } },
        { provide: getRepositoryToken(OrderEntity), useValue: { findOne: jest.fn(), save: jest.fn() } },
        { provide: getRepositoryToken(UserEntity), useValue: { findOneBy: jest.fn(), findOne: jest.fn() } },
        { provide: getRepositoryToken(AddressEntity), useValue: { findOneBy: jest.fn() } },
        { provide: getRepositoryToken(PromotionEntity), useValue: { findOne: jest.fn() } },
        {
          provide: DataSource,
          useValue: {
            transaction: jest.fn(),
          },
        },
        { provide: PaymentSuccessMailService, useValue: { sendPaymentSuccessMail: jest.fn() } },
        { provide: paymentFailMailService, useValue: { sendPaymentFailMail: jest.fn() } },
      ],
    }).compile();

    paymentService = module.get<PaymentService>(PaymentService);
    stockRepository = module.get(getRepositoryToken(StockEntity));
    orderRepository = module.get(getRepositoryToken(OrderEntity));
    userRepository = module.get(getRepositoryToken(UserEntity));
    addressRepository = module.get(getRepositoryToken(AddressEntity));
    promotionRepository = module.get(getRepositoryToken(PromotionEntity));
    dataSource = module.get<DataSource>(DataSource);
    mockPaymentSuccessMail = module.get<PaymentSuccessMailService>(PaymentSuccessMailService);
    mockPaymentFailMail = module.get<paymentFailMailService>(paymentFailMailService);
  });

  afterEach(() => {
    jest.clearAllMocks();
    process.env = originalEnv;
  });

  // VERIFICATION ORDER
  describe('verificationOrder', () => {
    const mockUser = { id: 1, mail: 'test@example.com', isVerified: true } as UserEntity;
    const mockCartDto: CartDto = {

```

```

    delivery_address_id: 10,
    billing_address_id: 20,
    items: [{ sku: 'ITEM-1', quantity: 2 }],
    promotion_code: '',
  });

  beforeEach(() => {
    userRepository.findOneBy.mockResolvedValue(mockUser);
    addressRepository.findOneBy.mockResolvedValue({});
    stockRepository.findOne.mockResolvedValue({ sku: 'ITEM-1', quantity: 10, product: { price: 50 } });
    orderRepository.save.mockResolvedValue({ id: 999 });
    jest.spyOn(paymentService, 'createCheckoutSession').mockResolvedValue({ url: 'http://stripe.com' } as any);
  });

  it('devrait jeter une NotFoundException si le stock/variant est introuvable', async () => {
    stockRepository.findOne.mockResolvedValue(null);
    await expect(paymentService.verificationOrder(mockCartDto, mockUser)).rejects.toThrow(NotFoundException);
  });

  it('devrait jeter une BadRequestException si le stock est insuffisant', async () => {
    stockRepository.findOne.mockResolvedValue({ sku: 'ITEM-1', quantity: 1, product: { price: 50 } });
    await expect(paymentService.verificationOrder(mockCartDto, mockUser)).rejects.toThrow(BadRequestException);
  });

  it('devrait appliquer une réduction en pourcentage et sauvegarder la commande', async () => {
    const cartWithPromo = { ...mockCartDto, promotion_code: 'REDUC20' };
    promotionRepository.findOne.mockResolvedValue({ name: 'REDUC20', minAmount: 50, promotionType: 'percentage', percentageValue: 20 });

    await paymentService.verificationOrder(cartWithPromo, mockUser);

    const savedOrderArg = orderRepository.save.mock.calls[0][0];
    expect(savedOrderArg.totalAmount).toBe(80);
    expect(paymentService.createCheckoutSession).toHaveBeenCalled();
  });

  // CREATE CHECKOUT SESSION
  describe('createCheckoutSession', () => {
    const mockItems = [
      { stock: { sku: 'ITEM-1', product: { name: 'T-shirt' } }, quantity: 2, priceAtPurchase: 50 },
    ] as any;

    it('devrait jeter une InternalServerErrorException si le nom du produit manque', async () => {
      const badItems = [{ stock: { sku: 'ITEM-1', product: {} }, quantity: 1, priceAtPurchase: 10 } ] as any;
      await expect(paymentService.createCheckoutSession(999, 1, badItems, 'test@example.com')).rejects.toThrow(InternalServerErrorException);
    });

    it('devrait créer une session Stripe avec succès', async () => {
      // @ts-ignore
      paymentService['stripe'].checkout.sessions.create.mockResolvedValue({ url: 'http://stripe-session.com' });
    });
  });

```

```

    const result = await paymentService.createCheckoutSession(999, 1, mockItems,
'test@example.com');

expect(paymentService['stripe'].checkout.sessions.create).toHaveBeenCalledWith(expect.objectContaining({
    customer_email: 'test@example.com',
    metadata: { orderId: '999', userId: '1' },
}));
expect(result).toEqual({ url: 'http://stripe-session.com' });
});

it('devrait intercepter les erreurs Stripe et jeter une InternalServerErrorException', async ()
=> {
    (paymentService['stripe'].checkout.sessions.create as jest.Mock).mockRejectedValue(new
Error('Stripe is down'));
    await expect(paymentService.createCheckoutSession(999, 1, mockItems,
'test@example.com')).rejects.toThrow('failed to create checkout session');
    });
});

// PAYMENT FAILED
describe('paymentFailed', () => {
    it('devrait jeter une ForbiddenException si la commande n'existe pas', async () => {
        orderRepository.findOne.mockResolvedValue(null);
        await expect(paymentService.paymentFailed(999, 1)).rejects.toThrow(ForbiddenException);
    });

    it('devrait annuler la commande si elle est PENDING et envoyer un mail', async () => {
        const mockOrder = { id: 999, status: OrderStatus.PENDING, user: { mail: 'test@test.com' } };
        orderRepository.findOne.mockResolvedValue(mockOrder);
        jest.spyOn(paymentService, 'sendMailPaymentFail').mockResolvedValue(undefined);

        await paymentService.paymentFailed(999, 1);

        expect(mockOrder.status).toBe(OrderStatus.CANCELLED);
        expect(orderRepository.save).toHaveBeenCalledWith(mockOrder);
        expect(paymentService.sendMailPaymentFail).toHaveBeenCalledWith(999, 'test@test.com');
    });
});

// WEBHOOK STRIPE
describe('constructEventWebhook', () => {
    let mockReq: any;
    let mockRes: any;

    beforeEach(() => {
        mockReq = { rawBody: 'raw_body_data' };
        mockRes = { status: jest.fn().mockReturnThis(), send: jest.fn(), json: jest.fn() };
    });

    it('devrait jeter une erreur 401 si la signature est invalide', async () => {
        (paymentService['stripe'].webhooks.constructEvent as jest.Mock).mockImplementation(() => {
            throw new Error('Invalid signature');
        });

        await paymentService.constructEventWebhook(mockReq, mockRes, 'bad_signature');
        expect(mockRes.status).toHaveBeenCalledWith(401);
        expect(mockRes.send).toHaveBeenCalledWith('webhook error: Invalid signature');
    });
});

```

```

describe('checkout.session.completed', () => {
  const mockEvent = {
    type: 'checkout.session.completed',
    data: { object: { metadata: { orderId: '999', userId: '1' } } },
  };

  it('devrait payer la commande, déduire le stock et envoyer un mail (Transaction OK)', async ()
=> {
    (paymentService['stripe'].webhooks.constructEvent as jest.Mock).mockReturnValue(mockEvent);

    const mockStock = { sku: 'ITEM-1', quantity: 10 };
    const mockOrder = { id: 999, status: OrderStatus.PENDING, items: [{ quantity: 2, stock:
mockStock }] };
    const mockEntityManager = {
      findOne: jest.fn().mockResolvedValue(mockOrder),
      save: jest.fn(),
    };

    dataSource.transaction.mockImplementation(async (cb: any) => cb(mockEntityManager));
    userRepository.findOne.mockResolvedValue({ id: 1, mail: 'success@test.com' });
    jest.spyOn(paymentService, 'sendMailPaymentSuccess').mockResolvedValue(undefined);

    await paymentService.constructEventWebhook(mockReq, mockRes, 'valid_signature');

    expect(mockOrder.status).toBe(OrderStatus.PAID);
    expect(mockEntityManager.save).toHaveBeenCalledTimes(1);
    expect(mockStock.quantity).toBe(8);
    expect(mockEntityManager.save).toHaveBeenCalledTimes(2);
    expect(paymentService.sendMailPaymentSuccess).toHaveBeenCalledTimes(1);
    expect(mockRes.status).toBe(200);
  });
});

describe('checkout.session.expired', () => {
  it('devrait annuler la commande', async () => {
    const mockEvent = {
      type: 'checkout.session.expired',
      data: { object: { metadata: { orderId: '999' } } },
    };

    (paymentService['stripe'].webhooks.constructEvent as jest.Mock).mockReturnValue(mockEvent);

    const mockOrder = { id: 999, status: OrderStatus.PENDING };
    orderRepository.findOne.mockResolvedValue(mockOrder);

    await paymentService.constructEventWebhook(mockReq, mockRes, 'valid_signature');

    expect(mockOrder.status).toBe(OrderStatus.CANCELLED);
    expect(orderRepository.save).toHaveBeenCalledTimes(1);
    expect(mockRes.status).toBe(200);
  });
});

// MAIL SERVICES
describe('Mail Sending Wrappers', () => {
  it('devrait appeler paymentSuccessMail', async () => {
    await paymentService.sendMailPaymentSuccess('test@test.com', 999);
  });
});

```

```
        expect(mockPaymentSuccessMail.sendPaymentSuccessMail).toHaveBeenCalledWith('test@test.com',
999);
    });

    it('devrait appeler paymentFailMail', async () => {
        await paymentService.sendMailPaymentFail(999, 'test@test.com');

        expect(mockPaymentFailMail.sendPaymentFailMail).toHaveBeenCalledWith(999, 'test@test.com');
    });
});
});
```